

High-Level Design (HLD) Guide — Volume 2

Tier 3 & Tier 4: data-heavy and scale-heavy systems (in plain language)

Target level: Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

About this volume

Volume 1 covered the twelve most common HLD problems (Tier 1 + Tier 2). **Volume 2 goes into the bigger, data-heavier systems** — the ones that come up in senior-leaning rounds or when the interviewer wants to see how you think about **huge scale, streaming data, and storage of large files**.

These systems sound intimidating ("design YouTube!"), but they're built from the **same building blocks** you already know: load balancers, queues, caches, sharded databases, and replication. This volume keeps the language **simple and explains every term as it appears**, so a scary-sounding system becomes a set of familiar pieces wired together.

Every problem answers the **same 15-point checklist** as Volume 1, so your interview muscle memory stays identical:

- | | |
|----------------------------|--|
| 1. Requirements | what it must do + how big/fast (functional + non-functional) |
| 2. Estimation | how much traffic and storage — with the math |
| 3. API style | REST vs gRPC vs GraphQL — and WHY |
| 4. Idempotency | do we need idempotency keys? why / why not |
| 5. Network protocol | HTTP, HTTP/2, WebSocket, gRPC — and why |
| 6. Database | SQL vs NoSQL — and WHY, based on how we read/write |
| 7. Data model | the tables / keys, explained |
| 8. Load balancer | L4 vs L7 vs API Gateway — which benefits we use |
| 9. Caching + data flow | how data moves between the DB and the cache |
| 10. Concurrency | where two things can clash, and how we prevent it |
| 11. Replication | do we need copies of the data? what kind? |
| 12. Multi-geo | serving users in many regions of the world |
| 13. Failure handling | what breaks, and how we keep working anyway |
| 14. Bottlenecks & scaling | where it strains first, and the fix |
| 15. Tradeoffs + follow-ups | the short, senior-sounding summary |

If a term shows up (like "sharding" or "consumer group"), we explain it in one plain line right where it's used — no assumed jargon.

What's inside (Tier 3 & Tier 4)

#	Problem	The one big idea
13	Web Crawler	Politely fetch billions of pages without repeating work

#	Problem	The one big idea
14	Typeahead / Autocomplete	Suggest top completions in a few milliseconds
15	Distributed Job Scheduler	Run tasks reliably, at the right time, exactly enough
16	YouTube / Netflix (video)	Store + stream huge video files smoothly
17	Google Drive / Dropbox	Sync files across devices without losing data
18	Payment System	Move money once, never twice, and keep a perfect ledger
19	Distributed Message Queue	Deliver events reliably between services (Kafka-style)
20	Search Engine / Log Search	Find matching documents fast in a giant text index
21	Metrics / Monitoring	Ingest and query billions of time-series data points
22	Ad Click Aggregator	Count clicks accurately on a firehose of events

A few plain-language building blocks used a lot in this volume

- **Sharding** = splitting one huge table across many machines, each holding a slice, so no single machine has to store or serve everything.
 - **Replication** = keeping extra copies of data on other machines, so if one dies you don't lose data and you can serve more reads.
 - **Message queue** = a durable "to-do list" between services. One service drops a task in; another picks it up later. It absorbs spikes and lets services fail independently.
 - **Stream processing** = doing math (counting, summing) on data *as it flows in*, instead of storing everything first and querying later.
 - **Object storage** (S3-like) = a cheap, near-infinite place to store big files (videos, images, backups), addressed by a key. Not a database — no queries, just get/put by key.
 - **CDN** (Content Delivery Network) = a global network of edge caches that serve files from a location near the user, so downloads are fast worldwide.
 - **Idempotency** = making a repeated request have the same effect as doing it once (so a retry can't, say, charge a card twice).
-

13. Web Crawler

The problem in one line. Starting from a handful of seed URLs, automatically discover and download billions of web pages — following the links inside them — without hammering any one website, without downloading the same page twice, and without getting stuck forever on a never-ending maze of links.

1. Requirements

Functional - Start from a list of **seed URLs** (a small starting set, e.g. `wikipedia.org`, `nytimes.com`) and crawl outward. - **Download** each page's HTML. - **Extract links** out of that HTML and add the new URLs to the list of pages still to crawl. - **Store the page content** somewhere a search engine (or other consumer) can later read it. - **Avoid re-crawling** a page we already have a fresh copy of. - **Respect `robots.txt`** — a text file a website publishes at `/robots.txt` that says which pages a bot is and isn't allowed to fetch — and be **polite** (don't send so many requests to one site that we effectively attack it).

Non-functional - **Massive scale**: billions of pages, trillions of links discovered over time. - **Politeness over speed**: it is far more important to never overload a single website than to crawl as fast as physically possible. A crawler that gets a site's IP address banned has failed, even if it was "fast." - **Eventual freshness, not real-time**: pages get re-crawled every so often (days to weeks) to catch updates — we don't need sub-second freshness. - **Fault tolerant**: with billions of pages and thousands of machines running for weeks, some machine will always be crashing, some website will always be timing out. The system must keep making forward progress anyway. - **No public-facing traffic**: unlike a URL shortener, nobody outside our own company calls this system directly — it just runs continuously in the background (more on this in section 3).

2. Estimation

Target: crawl 10 billion pages, and re-crawl the "live" web every ~2 weeks.

Pages per second needed:

$10,000,000,000 \text{ pages} / (14 \text{ days} * 86,400 \text{ sec/day}) \approx 8,300 \text{ pages/sec average}$
(design for a few times that at peak, so ~25,000 pages/sec)

Average page size (HTML + assets we keep) $\approx 100 \text{ KB}$

Bandwidth: $8,300 \text{ pages/sec} * 100 \text{ KB} \approx 830 \text{ MB/sec}$ sustained download

Storage: $10\text{B pages} * 100 \text{ KB} \approx 1 \text{ PB (petabyte)}$ of raw content
(compresses well $\rightarrow$ realistically a few hundred TB)

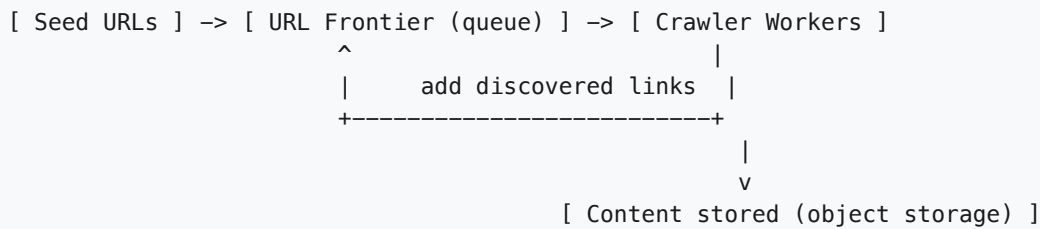
URL frontier (the "to crawl" queue, explained in section 7):

at any time it might hold billions of discovered-but-not-yet-fetched URLs. Each entry is small (~100 bytes of metadata) $\rightarrow$ a few hundred GB, easily fits a distributed queue/KV store, not a single machine.

Worker fleet: if one worker machine can sustain ~50 pages/sec (network and parsing bound, not CPU bound), $25,000 \text{ pages/sec} / 50 = \sim 500$ worker machines at peak. That's a fleet, not a single box — a big signal that this system must be horizontally distributed from day one.

3. API style — REST vs gRPC vs GraphQL

Choice: no public API at all — this is an internal, queue-driven pipeline.



A URL shortener or a payment system has an outside caller (a browser, a merchant) that needs a documented request/response contract — that's what REST/gRPC/GraphQL are for. A crawler has **no external caller**. Its "input" is a queue of URLs and its "output" is stored pages that other internal systems (like a search index, see problem #20) read later. So instead of an API, the important interface is the **message queue / task queue** between the frontier and the workers, plus internal **gRPC** calls between our own services (e.g., a worker asking a "robots.txt cache" service "am I allowed to fetch this URL?"). We mention gRPC here only because it's a good fit for fast, typed, internal service-to-service calls — not because crawling itself is request/response.

4. Idempotency — needed?

Yes, and it comes almost for free from deduplication.

"Idempotent" means: doing the same operation twice has the same effect as doing it once. If a worker crashes half-way through processing a URL and another worker picks up that same URL again, that must be **harmless**, not a bug.

- Because we always **overwrite** the stored content for a given URL (keyed by the URL, see section 7) rather than appending, fetching the same URL twice just writes the same content twice — no duplicate side effects.
- The **deduplication / "seen" set** (section 10) is what actually prevents most double-work: once a URL has been crawled recently, it's marked as seen and won't be re-queued. So the system is naturally idempotent by design — we don't need a client-style "idempotency key" like a payment API would; the URL itself already *is* the natural idempotency key.

5. Network protocol

Two different protocols for two different hops: - **Crawler worker** → **target website**: plain **HTTP/HTTPS**, exactly like a browser. We must follow redirects, respect `Content-Type`, and honor timeouts, because we're talking to millions of servers we don't control, with wildly different speeds and reliability. - **Frontier** ↔ **workers (internal)**: a **message queue** (e.g., Kafka or a managed queue service) rather than a direct call — a queue is a durable "to-do list" between two parts of a system: one side drops work in, the other picks it up later. This lets us add or remove worker machines freely without the frontier needing to know who's currently listening.

There's no need for WebSockets here — nothing needs a persistent, two-way, always-open connection; each page fetch is a simple one-shot request/response.

6. Database — SQL vs NoSQL

We actually need **three different stores**, each chosen for a different access pattern — this is a great thing to say out loud in an interview, because it shows you're not forcing one database to do three jobs.

1. **URL metadata + crawl state → a scalable key-value / wide-column NoSQL store** (e.g., DynamoDB, Cassandra, or Bigtable-style). Why: the access pattern is "look up or update one URL's state by its URL/key" at huge volume (billions of rows), with no joins needed. That's the textbook case for a KV store, not a relational database.
2. **Page content (the actual downloaded HTML/text) → object storage** (S3-like — a cheap, near-infinite place to store big files by key, not a database with queries). Why: page bodies are large blobs (tens to hundreds of KB each) and we only ever fetch them by key ("give me the content for this URL"). Object storage is far cheaper per GB than a database and is built exactly for this "write once, read by key" pattern.
3. **The "have I seen this before?" set → a Bloom filter backed by / paired with a fast KV store.** A **Bloom filter** is a compact, memory-efficient structure that answers "have I probably seen this value before?" — it can say "definitely no" for certain, or "probably yes" with a small, tunable chance of being wrong (a false positive). We use it because checking "have I crawled this exact URL recently?" billions of times per day against a normal database would be far too slow and expensive; checking a Bloom filter is essentially free (a handful of memory lookups). We accept the rare false positive (skipping a page we hadn't actually crawled yet) as a reasonable trade for enormous speed and low memory use.

Why not just SQL for everything: a relational database's strengths — joins, multi-row transactions — aren't needed here. The dominant operations are single-key lookups and bulk sequential writes/reads, at a scale (billions of rows, petabytes of blobs) where a single SQL instance would fall over long before a horizontally-sharded KV store or object storage would.

7. Data model (detailed)

Table: urls (KV / wide-column, partitioned by domain — see section 8)

url	(STRING)	PARTITION KEY (grouped by domain, see below)
domain	(STRING)	
status	(ENUM)	PENDING IN_PROGRESS DONE FAILED
priority	(INT)	higher = crawl sooner (e.g. popular sites)
last_crawled	(TIMESTAMP)	when we last successfully fetched it
depth	(INT)	how many hops from a seed URL (crawl-trap guard)
content_hash	(STRING, nullable)	hash of the page body, for near-duplicate detection

Object storage: content bucket

key = hash(url) e.g. "a93f...:page.html"
value = the raw downloaded page (often gzip-compressed)

Bloom filter / seen-set (in-memory + persisted snapshot)

input: url → "have we crawled this in the last N days?"

Worked example: we crawl `https://news.example.com/story/42`. 1. Compute `domain = news.example.com`. 2. Check the Bloom filter: "have I seen this URL recently?" If yes, skip it entirely —

no network call at all, which is the whole point of the filter. 3. If not seen: fetch the page over HTTP, get back the HTML. 4. Store the HTML in object storage under a key derived from the URL's hash. 5. Update the `urls` row: `status = DONE`, `last_crawled = now`, `content_hash = <hash of body>`. 6. Extract every `<a href="...">` link inside the HTML, and for each new-looking link, insert a new row with `status = PENDING` into the `urls` table / frontier queue. 7. Add this URL to the Bloom filter so we don't repeat step 2's "not seen" answer for a while.

8. Load balancer — L4 vs L7 vs API Gateway

There's no public traffic to balance here (see section 3), so the interesting "distribution" problem isn't a classic load balancer in front of users — it's **how we spread work across worker machines while keeping politeness rules per website**.

```
[ URL Frontier, partitioned BY DOMAIN ]
domain shard A    domain shard B    domain shard C
      |              |              |
[ workers ]      [ workers ]      [ workers ]
(pull URLs for   (pull URLs for   (pull URLs for
domains in      domains in      domains in
shard A only)   shard B only)   shard C only)
```

Key design decision: the frontier is partitioned by domain, not randomly. If URLs for `nytimes.com` could be picked up by any of our 500 workers at once, we'd have no single place enforcing "only one request to nytimes.com every N seconds" — that's the politeness rule from section 1. By routing all of one domain's URLs to the same shard (and therefore the same small set of workers, or through a domain-level rate limiter), one place always knows "when did we last hit this domain?" This is conceptually similar to consistent hashing (a way of assigning keys to shards so that adding/removing a shard reshuffles as few keys as possible) applied to domains instead of user IDs.

There's still a thin **L7 layer (an internal API gateway)** in front of the small number of *internal control-plane* calls (e.g., a worker asking "give me my next batch of URLs" or "what does `robots.txt` say for this domain?") — L7 is useful there because it can route by path/service the way a normal microservice mesh would. But there is no L4/L7 balancing of *outbound* crawl traffic — that's fundamentally different from the inbound-traffic problem other systems in this guide solve.

9. Caching + data flow (DB ↔ cache)

Two caches matter a lot here, and both exist to avoid slow, repeated network round trips:

1) DNS cache

Every fetch needs "what IP address is nytimes.com?" (a DNS lookup). DNS lookups are surprisingly slow (tens of milliseconds) and we do them constantly -> cache hostname -> IP mappings locally on each worker (respecting the DNS record's own TTL, "time to live").

2) robots.txt cache

Before crawling ANY page on a domain, we must fetch and obey that domain's /robots.txt (which pages are off-limits, and sometimes a suggested crawl delay). Re-fetching robots.txt before every single page would double our request count for no benefit, since it rarely changes -> fetch it once per domain, cache it (e.g., for 24 hours), and consult the cached copy for every subsequent page on that domain.

Both caches follow the same shape: **fetch once, reuse many times, expire on a timer**. The `urls` KV table is the source of truth for crawl state; the caches above never hold anything that isn't safe to lose and re-fetch — if a worker restarts with a cold cache, it just re-fetches DNS and robots.txt once, at a small cost, and carries on.

10. Concurrency

Three different races can happen, and each needs its own guard:

- **Two workers crawling the same URL at once.** We prevent this with a **lease** — a temporary, time-limited claim on a piece of work. A worker asks the frontier for a URL, and the frontier marks that row `IN_PROGRESS` with an expiry time (e.g., "leased for 5 minutes"). No other worker can take a leased URL. If the worker finishes, it marks the URL `DONE`. If it doesn't finish in time (crash, hang), the lease **expires automatically** and the URL goes back into the pool for someone else to try — this is also our main answer to worker crashes in section 13.
- **Politeness across many workers on the same domain.** Even with domain-partitioned shards (section 8), multiple workers might serve one busy domain. A simple **token bucket rate limiter per domain** (a counter that refills at a fixed rate and each request "spends" one token) enforces "no more than 1 request every 2 seconds to this domain," shared across whichever workers are touching that domain right now.
- **Duplicate discovery of the same link.** Two different pages might both link to the same URL at nearly the same moment, and two workers might both try to insert it into the frontier. We guard this with a **"insert if not already present"** conditional write against the Bloom filter / seen-set (much like the URL shortener's "insert if short code doesn't exist" pattern) — whichever insert loses the race is simply a no-op.

11. Replication — needed?

Yes, for both pieces of state, though for different reasons.

- **The frontier and the seen-set** are replicated (multiple copies, e.g. 3 copies) so that a single machine failure doesn't lose track of "what still needs crawling" or "what we've already crawled." Losing the frontier would mean re-discovering the entire web from seed URLs again — extremely wasteful — so durability here matters a lot.

- **Page content in object storage** is replicated by the object store itself (that's a built-in feature of S3-like systems) so a disk failure doesn't lose downloaded pages that cost real time and bandwidth to fetch.
- Unlike the URL shortener, this data does change over time (crawl status, freshness), so we can't lean on "immutability makes staleness harmless" the same way — but a *few seconds* of replica lag on crawl state is still totally fine, since re-crawling a page a little early or late has no user-facing consequence.

12. Multi-geo

Crawlers placed in multiple regions, close to the sites being crawled:

```
[ Crawlers in US region ]    -> fetch US-hosted sites (low latency,
                               fewer cross-ocean hops)
[ Crawlers in EU region ]    -> fetch EU-hosted sites
[ Crawlers in APAC region ]  -> fetch APAC-hosted sites
```

Frontier + seen-set can stay in one or a few regions (they're metadata, small and centrally coordinated); it's the WORKERS that benefit most from being geographically close to the websites they fetch.

The reasoning here flips compared to most systems in this guide: multi-geo isn't about being close to *our users* (there are no end users hitting this system directly) — it's about being close to the **websites we're downloading from**, since network latency to a far-away server directly slows down every single fetch, multiplied across billions of pages.

13. Failure handling

Worker crashes mid-fetch	-> its lease (section 10) expires -> the URL automatically returns to the frontier for another worker to try.
A single website is down/slow	-> per-request TIMEOUT (e.g., 10s); mark that URL FAILED, retry later with backoff; never let one slow site stall a worker forever.
Crawler trap	-> some sites generate infinite URLs (e.g. a calendar with a "next month" link forever, or session IDs in every URL). Guard with a MAX DEPTH (stop following links past N hops from a seed) and a MAX URLS PER DOMAIN, so one bad site can't consume the whole crawl budget.
Bloom filter false positive	-> we skip a page thinking we've already crawled it, even though we haven't; acceptable, rare, and self-corrects on the next scheduled re-crawl pass.
Frontier shard down	-> replicas (section 11) take over; the domains on that shard are briefly delayed, not lost.
Entire region down	-> other regions' crawlers keep working; that region's share of the web is crawled a bit later, not never.

The overriding design principle: **a crawler is a long-running background job, not a user-facing request** — so the right failure response is almost always "retry later, keep moving," not "fail loudly and stop."

14. Bottlenecks & scaling

- **One very popular domain** (e.g., a huge news site with millions of pages) can't be crawled faster just by adding more workers, because politeness caps how fast *that one domain* can be hit — the fix is patience (spread its crawl over a longer window), not more machines.
- **Frontier size** (billions of pending URLs) → the frontier itself must be a sharded, distributed queue/KV store, not a single in-memory queue — this is why we partitioned it by domain in section 8 rather than keeping it all in one place.
- **Bandwidth** is often the real ceiling, not CPU — fetching and storing hundreds of megabytes per second sustained is a network and storage-throughput problem, which is why worker count in section 2 was sized off pages/sec, not raw compute.
- **Parsing HTML and extracting links at scale** is CPU work that scales embarrassingly well (each page is independent) — just add more worker machines.
- **Storage growth** (a petabyte-scale content store) → object storage scales horizontally by design, and old/unreachable pages can be moved to cheaper "cold" storage tiers over time.

15. Tradeoffs & what to say

"This is a background pipeline, not a request/response API, so there's no REST/gRPC surface to design for external callers — the real interfaces are the URL frontier queue and the object store. The frontier is partitioned by domain so politeness (rate limiting per site, respecting robots.txt) is always enforced in one place. A Bloom filter gives a cheap, approximate 'have I seen this?' check so we don't re-crawl the same URL over and over; a content hash catches near-duplicate pages. Page metadata lives in a sharded KV store, because it's all single-key lookups at huge volume; page bodies live in cheap object storage, because they're big blobs read by key. Leases make crashed-worker recovery automatic, and depth/URL limits stop crawler traps from consuming the whole budget. Because the write side has no user waiting on it, the system optimizes for steady, polite, fault-tolerant throughput over raw speed."

Follow-ups an interviewer may probe - How do you decide crawl *priority* (which pages to fetch first)? (Score by estimated importance — e.g., how many other pages link to it, similar to the idea behind PageRank — and by how stale the last copy is.) - How do you detect near-duplicate pages, not just identical URLs? (Hash the page content, or parts of it, and compare hashes — two different URLs with the same hash are likely duplicates or mirrors.) - What stops us from crawling the *entire* internet forever on one pass? (We don't try to — we run continuous passes with priority and freshness scores, so important pages get re-crawled often and obscure ones rarely.) - How would you crawl JavaScript-heavy pages that need a real browser to render content? (A smaller, more expensive pool of "headless browser" workers handles just those sites, separate from the fast plain-HTTP worker pool.)

14. Typeahead / Autocomplete

The problem in one line. As a user types a few letters (a "prefix"), instantly show the top ~5-10 most popular completions — like Google search suggestions.

1. Requirements

Functional - User types a prefix, e.g. "goo". System returns the top-K (say 5-10) most popular full search terms that start with "goo" — e.g. "google", "good morning", "google maps". - Suggestions are ranked by **popularity** (how often people search that term), not alphabetically. - Suggestions update over time as new terms trend and old ones fade.

Non-functional - **Extremely low latency** — this must feel instant, ideally under ~50-100 ms, because it fires on every keystroke. - **High read volume** — every keystroke of every user is a lookup. Far more reads than writes. - Slightly **stale** suggestions (a few hours/day old) are totally fine. Nobody notices if yesterday's top result is missing this morning's breaking news. - Suggestions may differ by **country/language** (locale-aware).

2. Estimation

```
Assume 500M searches/day across the product.
  search QPS ~ 500M / 86400 ~ 5,800/sec (peak ~3x = ~17,000/sec)
Typeahead is WORSE than search QPS: a user typing "google" fires ~6
lookups (one per keystroke), so:
  suggest QPS ~ 6 * search QPS ~ 35,000/sec (peak higher)
Data size: assume 50M distinct search terms worth keeping.
  each term ~50 bytes (text + count) -> 50M * 50B = 2.5 GB
-> small enough to hold ENTIRELY in memory. This one fact drives
    most of the design: it's a caching/in-memory problem, not a
    big-storage problem.
```

3. API style — REST vs gRPC vs GraphQL

Choice: REST for the public-facing call; gRPC is fine internally if the suggestion service is called by other backend services.

```
GET /suggest?q=goo&locale=en-US
-> { "suggestions": ["google", "good morning", "google maps", ...] }
```

Why REST: a plain REST call — GET with a query string — is the perfect fit here: one tiny endpoint, one tiny JSON array back. No need for GraphQL's flexible field selection (the response shape never varies) or gRPC's streaming (there's nothing to stream, it's a single quick request/response). **The real challenge isn't the API style — it's shaving off every millisecond**, since this call sits directly in the user's typing path.

4. Idempotency — needed?

Reads: yes, trivially. GET /suggest?q=goo is a **read** — calling it twice never changes anything, so it's naturally idempotent (this term just means "safe to repeat without side effects").

Writes: append-only, no conflict to resolve. Every time a user submits a search, we log that event ("someone searched for 'google maps' at time T"). Logging is just an **append** — adding one more line to a log — never an update-in-place, so there's nothing to retry-and-collide on. Duplicate log lines (from a client retry) just slightly overcount one term's popularity, which is harmless at this scale.

5. Network protocol

HTTPS over HTTP/2. Plain request/response, no server push needed, so WebSocket (a persistent two-way connection) would be overkill. One important **client-side** trick: **debouncing** — wait ~100-200 ms after the last keystroke before firing the request, so a fast typist doesn't fire 6 requests for 6 letters, only 1-2.

6. Database — SQL vs NoSQL vs in-memory

Choice: split it in two. - **Query path (the read that must be instant): an in-memory structure / Redis.**

The entire set of "prefix -> top suggestions" fits in a few GB (from Section 2), so keep it all in RAM.

Nothing beats RAM for latency. - **Build path (computing popularity from raw searches): a big append-friendly log store** (e.g., Kafka -> data lake / HDFS) holding raw "who searched what" events, processed by a **batch job** (like Spark) or a streaming job.

Why not a traditional SQL/NoSQL database for the query path: even a fast key-value store call over the network (~1-5 ms) eats into our tight latency budget when it fires on every keystroke. An in-process or same-datacenter in-memory cache is faster still. **Why not compute suggestions synchronously on each request:** ranking "which terms starting with 'goo' are most popular" over billions of logged searches is far too slow to do live — so we precompute it offline and just **serve** the precomputed answer on the read path.

7. Data model (detailed)

The centerpiece data structure is a **trie** — a "prefix tree," where each step down the tree matches one more typed letter. Every node represents "the string formed by the letters from the root down to here." A node can cache its own **top-K list**: the K most popular full terms that start with that prefix. That's what makes a lookup basically instant — no searching, just "walk down one node per typed letter, then read the cached list."



Node for prefix "goo" caches:
 topK = ["google", "good morning", "google maps",
 "good", "goodreads"]
 -> GET /suggest?q=goo just walks root->g->o->o and
 returns that cached list. No further computation.

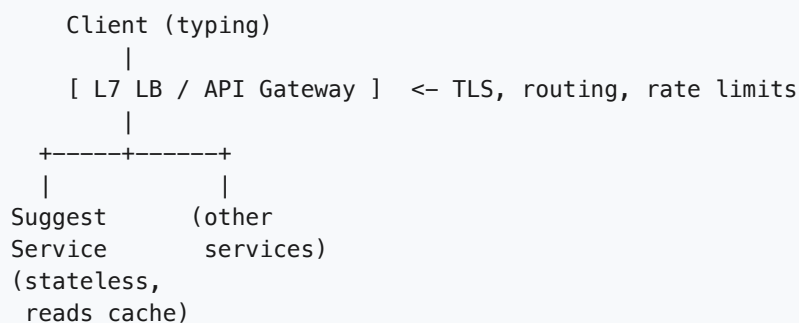
An alternative (simpler to reason about, same idea): skip the tree walk and store a flat key-value map, **prefix -> topK list**, e.g. "goo" -> ["google", "good morning", ...]. This is easier to shard and cache (it's just a KV lookup), at the cost of storing every prefix explicitly rather than sharing tree nodes. Both are valid; mention the tradeoff.

Raw log record (append-only, the BUILD path's input):

```

-----
query_text  (STRING)      e.g. "google maps"
user_id     (STRING)
timestamp   (TIMESTAMP)
locale      (STRING)
-----
  
```

8. Load balancer — L4 vs L7 vs API Gateway



Choice: L7 / API Gateway. The suggestion service is small, stateless, and read-only, so routing needs are simple — mostly just "get this request to any healthy instance fast." L7 still earns its keep for TLS termination and **rate limiting** (one abusive client shouldn't be able to hammer every keystroke as a separate uncapped request). No need for GraphQL-style gateway logic; this is the simplest kind of service to load-balance.

9. Caching + data flow (DB ↔ cache)

This problem is *mostly* about the cache — there almost isn't a separate "database" on the hot path. Two clearly separate flows:

```
QUERY PATH (must be milliseconds):
  User types "goo"
    -> Suggest Service
    -> look up "goo" in in-memory trie / Redis
    -> return cached topK list
  (No disk read, no computation — just a lookup.)

BUILD PATH (offline, runs periodically, e.g. hourly):
  Raw search logs (Kafka) -> batch/stream job
    -> counts how often each full term was searched
    -> for every prefix, computes topK most popular completions
    -> writes a brand-new trie / KV snapshot
    -> ATOMICALLY SWAPS it in for the Suggest Service
        to start reading (old version simply discarded)
```

The **read side never talks to the raw logs**. It only ever reads the latest precomputed snapshot. This separation — cheap instant reads, expensive periodic rebuild — is the single most important idea in this design.

10. Concurrency

- The query path is **almost entirely reads** — many users reading the same cached data at once is not a conflict, it's the ideal case for a cache.
- The build path runs **offline, in its own job**, not competing with live traffic for the same resources.
- **Swapping in a new trie** while live traffic is reading the old one: build the new version fully in the background, then **atomically flip a pointer** ("start reading from this new snapshot now") — readers never see a half-built, inconsistent structure.
- Logging a search event is a simple **append** (add to a queue/log) — many users doing this concurrently just means many independent appends, no shared state to fight over.

11. Replication — needed?

Yes, heavily — this is a read-heavy system. - Replicate the in-memory suggestion cache across **many instances** of the Suggest Service, so read load spreads across all of them (each instance holds its own full copy — remember, the whole dataset is only a few GB). - Because suggestions are allowed to be **a little stale**, replicas don't need to be perfectly in sync at every instant — each just needs the latest *rebuilt* snapshot, applied whenever convenient. - The raw log store (Kafka / data lake) is also replicated for durability, but that's a background concern, not on the user-facing path.

12. Multi-geo

Build job (runs once, e.g. per region or globally)

|
v

New suggestion snapshot

|

| | |
Region US Region EU Region APAC
(in-memory copy in each region)

Copy the whole suggestion snapshot to every region so users always read from the nearest one — this is a read, so we want it as close to the user as possible. Suggestions can also be **locale-specific**: build a separate trie/topK set per language or country (e.g., "goo" in English vs. in French), and route each user to the version for their locale.

13. Failure handling

In-memory cache instance crashes	-> LB routes to a healthy replica instance (has its own full copy already).
Whole cache tier degraded	-> fall back to a smaller, simpler in-memory set (e.g. global top-1000 terms with no personalization) rather than fail the request.
Build job fails mid-run	-> the OLD snapshot just keeps serving; we simply retry the build next cycle. Stale is fine, broken is not.
Suggest Service fully down	-> client-side: just don't show suggestions; the search box itself still works.

Design principle: **suggestions are a nice-to-have layered on top of search, never a hard dependency**. Always fail toward "show nothing" or "show something slightly stale," never toward blocking the user from searching.

14. Bottlenecks & scaling

- **Read QPS is the main load** (Section 2 showed it's higher than raw search QPS, since every keystroke counts) → horizontally scale stateless Suggest Service instances behind the LB; each holds a full in-memory copy, so there's no shared bottleneck.
- **Build job cost** grows with total search volume, not with read QPS → it runs on its own schedule (e.g., hourly) and its cost is decoupled from live traffic.
- **Hot prefixes** (everyone types "a", "the", common letters first) are naturally handled — those are exactly the entries that get cached, and every replica already has them.
- **Debouncing on the client** (Section 5) meaningfully cuts read QPS for free — worth mentioning as a cheap scaling lever.

15. Tradeoffs & what to say

"I split this into two paths. The **query path** just walks a trie — or does a flat prefix-to-topK lookup — entirely in memory, so it's a few milliseconds: no database call on the hot path at all. The **build path** is a separate offline batch/stream job that aggregates raw search logs into popularity counts, computes each prefix's top-K, and atomically swaps in a fresh snapshot periodically. Because stale suggestions are acceptable, I can replicate the whole snapshot to every instance and every region, so reads are always local and fast. REST is fine since the payload is tiny; the real work is keeping the read path free of anything slow. Failure just means falling back to a smaller, simpler suggestion set — never blocking the user from typing or searching."

Follow-ups an interviewer may probe - How to personalize suggestions per user? (Blend the global topK with the user's own recent searches, computed client-side or in a thin per-user layer.) - How to make brand-new trending terms show up faster than the next scheduled rebuild? (Run a lightweight streaming "hot terms" job on a much shorter cycle, merged on top of the slower batch-built base set.) - How to handle typos / fuzzy prefixes? (Separate concern — usually a fuzzy-matching layer in front of, or alongside, the exact-prefix trie.) - Why not just query a search index (like Elasticsearch) live for every keystroke? (Too slow under this QPS and latency budget — precomputing and caching wins.)

15. Distributed Job Scheduler

The problem in one line. Let users submit "background jobs" — run this now, run this at a specific time, or run this every day at 2am — and reliably make sure each job actually runs, even if a machine crashes in the middle.

1. Requirements

First, what is a "job"? A job is just a unit of work the system runs for you later, instead of right now — for example "send this email," "generate this report," "resize this image."

Functional - Submit a job to run **immediately** (as soon as a worker is free). - Submit a job to run **at a specific time** ("run at 2026-08-01 09:00 UTC"). - Submit a **recurring** job using a cron expression ("every day at 2am"). - The system must actually **run** the job on some machine, called a **worker**. - If a job fails, **retry** it a few times before giving up. - Let the user **check status**: pending, running, succeeded, failed.

Non-functional - **Reliability over speed**. It is fine if a job runs a few seconds late. It is *not* fine if a job silently never runs, or if a critical job (like "charge the customer") runs twice and charges them twice. -

At-least-once execution — explained fully in section 4 — is the realistic guarantee. - **Scale**: millions of jobs a day, many of them recurring. - **Isolation**: one slow or crashing job type should not block other jobs.

2. Estimation

```
Assume 10M jobs/day submitted across all job types.  
jobs/sec average = 10M / 86400 ~= ~115 jobs/sec  
peak (3x average, e.g. everyone's "daily 2am" jobs firing together)  
  ~= 350 jobs/sec at peak  
  
Job record size: ~1 KB (type, payload, timestamps, status).  
10M/day * 1 KB = 10 GB/day of job metadata  
Keep "done" jobs for 30 days for status lookups -> ~300 GB (fine for SQL,  
or archive old ones to cold storage / a data warehouse).  
  
Worker capacity: if an average job takes 2 seconds of CPU time,  
350 jobs/sec * 2 sec = 700 concurrent job-seconds needed  
-> roughly 700 worker "slots" needed at peak (spread across many  
worker machines, each running many jobs concurrently).
```

The exact numbers matter less than the exercise: it tells you whether a single database can hold all job metadata (yes, easily) and roughly how many workers you need (a few hundred to a few thousand machines, depending on job size — comfortably horizontal).

3. API style — REST vs gRPC vs GraphQL

Choice: REST for the public submit/query API. gRPC internally between the queue and workers (or just the queue's native client protocol).

```
POST /api/v1/jobs
  { type: "send_email", payload: {...}, run_at?: "...", cron?: "0 2 * * *",
    idempotency_key: "email-123" }
-> { job_id, status: "pending" }

GET /api/v1/jobs/{job_id}  -> { status, attempts, last_error, ... }
DELETE /api/v1/jobs/{job_id} -> cancel a not-yet-run job
```

Why REST: submitting a job and checking its status are simple, independent resource operations — create a job, read a job. There's no deeply nested data to shape (GraphQL's strength) and no need for bidirectional streaming (gRPC's strength) on the *public* side. **Why gRPC internally:** once a worker picks up a job, worker-to-worker or worker-to-coordination-service calls are high-frequency and latency-sensitive, so a typed, low-overhead protocol like gRPC pays off. If a message queue (Kafka, SQS) is used, workers just speak that queue's client protocol instead of gRPC.

4. Idempotency — needed?

Yes — this is the single most important idea in this whole design.

Quick definition: an operation is **idempotent** if doing it twice has the same effect as doing it once. "Set my balance to \$100" is idempotent. "Add \$100 to my balance" is not — do it twice and you've added \$200.

Why does this matter here? Because of the guarantee we chose in the next section: **at-least-once execution**. That means our system may, in rare cases (a worker crash right after finishing a job but before reporting success), run the **same job twice**. We cannot make this impossible without a lot of extra complexity (see "exactly-once" below), so instead we make it **harmless**:

- Every job carries an **idempotency_key** — a string that uniquely identifies the *intent* ("charge order #123", not "run job #4821"). Before running, the worker (or the job's own business logic) checks: "have I already completed this key?" If yes, skip / return the cached result. If no, do the work and record the key as done.
- Design job payloads to be idempotent by nature where possible: "set the invoice status to PAID" rather than "increment the paid counter."

At-least-once vs exactly-once, explained simply:

```
At-least-once: the system guarantees the job runs ONE OR MORE times.
Easy to build. Requires idempotent jobs to be safe.
Exactly-once:  the system guarantees the job runs EXACTLY one time, ever.
Very hard in a distributed system (would need a perfectly reliable,
crash-proof handoff between "mark done" and "the effect happened").
Most real systems fake it by doing at-least-once + idempotency, which
gives the same *user-visible* result with far less complexity.
```

Recommendation: at-least-once delivery + idempotent jobs. Say this pairing out loud in the interview — it's the standard, expected answer.

5. Network protocol

HTTPS (TLS) for the public submit/status API — plain request/response, no need for a persistent connection since a client just submits a job and later polls or gets a webhook.

Internally, the scheduler-to-queue and queue-to-worker hops use whatever the queue provides: Kafka uses its own binary protocol over TCP, SQS uses HTTPS calls, a database-backed queue uses the DB's normal connection. Workers typically **pull** (long-poll or subscribe) rather than have jobs pushed to them — this naturally limits how much work a worker takes on at once (it only asks for more when it's ready), which is a form of back-pressure.

6. Database — SQL vs NoSQL

Choice: SQL (Postgres/MySQL) for job metadata and schedules. A message queue (Kafka or SQS) — or the same SQL database, at smaller scale — for the "ready right now" queue.

Why SQL for metadata: picking up a due job is a classic **claim** operation — "find one job that is due and not yet claimed, and atomically mark it as claimed by me." This needs a **transaction** (a group of operations that either all happen or none do) so that two schedulers can't both grab the same job. SQL gives us `SELECT ... FOR UPDATE SKIP LOCKED` or an atomic `UPDATE ... WHERE status='pending' RETURNING *`, which is exactly this kind of safe, all-or-nothing claim. NoSQL key-value stores can do single-row atomic updates too, but querying "all jobs due before now" efficiently is a range query SQL indexes handle naturally.

Why a separate queue for "ready now" jobs: once a scheduler decides a job is due, we don't want workers polling the database directly at high frequency (that would hammer the DB). So the scheduler pushes the job onto a message queue, and workers pull from *that* instead. A message queue is built exactly for this: durable, ordered-enough, and handles many consumers pulling in parallel far more cheaply than a database would. At small scale, skipping the extra queue and having workers poll the DB directly is a reasonable simplification — say this tradeoff aloud.

```
[ Job DB (SQL) ] <-- stores every job, its schedule, its status
      |
[ Scheduler ]  polls DB for "due" jobs, claims them (transaction),
      |          and pushes them onto...
      v
[ Queue (Kafka/SQS) ] <-- the "ready to run right now" list
      |
[ Worker pool ] pulls jobs off the queue, runs them, reports status
      |                  back into the Job DB.
```

7. Data model (detailed)

Table: jobs

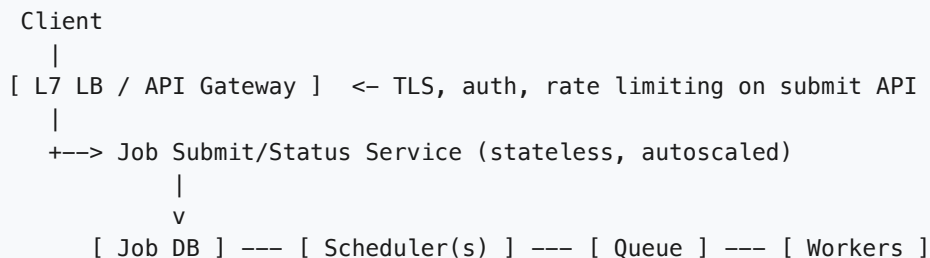
id	(UUID)	PRIMARY KEY
type	(STRING)	e.g. "send_email", "generate_report"
payload	(JSON/BLOB)	the input data the job needs
run_at	(TIMESTAMP)	when a one-off job should run
cron	(STRING, null)	e.g. "0 2 * * *" for recurring jobs
status	(ENUM)	pending claimed running succeeded failed dead_letter
attempts	(INT)	how many times we've tried
max_attempts	(INT)	give up after this many
next_retry_at	(TIMESTAMP)	when to try again after a failure
idempotency_key	(STRING)	unique intent key (see section 4)
locked_by	(STRING, null)	which worker/scheduler claimed it
locked_at	(TIMESTAMP)	when it was claimed (for lease expiry)
last_error	(STRING, null)	for debugging failed jobs

Index: (status, run_at) -> fast "find due jobs" query

Unique index: idempotency_key -> prevents duplicate submissions

For recurring jobs, a common pattern is to keep **one row per cron schedule**, and each time it fires, the scheduler either updates `run_at` to the next occurrence or writes a fresh "job run" row into a separate `job_runs` table (so history of each firing is kept). Either works; the row-per-run version makes status/history queries simpler.

8. Load balancer — L4 vs L7 vs API Gateway



Choice: L7 / API Gateway in front of the public submit/status service. This is the only place a traditional load balancer is needed — it's a normal stateless HTTP service, so L7 gives us path routing, TLS termination, auth, and rate limiting (so one client can't flood the system with job submissions).

Workers do not sit behind a load balancer. They *pull* jobs from the queue rather than having requests routed to them, so there's nothing to balance — the queue itself spreads work evenly because every idle worker is competing to pull the next item.

9. Caching + data flow (DB ↔ cache)

There isn't much to cache here — this system is not read-heavy in the URL-shortener sense. The "queue" is the important data-flow structure, not a cache. Two small caching wins:

Status lookups: if a user's dashboard polls "is my job done?" frequently, cache the (job_id -> status) pair for a few seconds in Redis to avoid hitting the DB on every poll. Short TTL because status changes.

Scheduler's "due jobs" query: instead of querying the DB every second, the scheduler can keep a small in-memory min-heap of "next due job per shard" so it only re-queries the DB when that heap says something is imminent. This isn't a cache exactly, but the same idea: avoid redundant DB hits for a value that rarely changes moment to moment.

The real "data flow" to describe in the interview is: **Job DB -> Scheduler -> Queue -> Worker -> back to Job DB**. Each arrow is a hop where a job's status changes, and each hop is where failures can happen (covered in section 13).

10. Concurrency

This is where the classic race condition of this problem lives: **two schedulers (or two workers) must never run the same job at the same time**.

THE RACE:

Scheduler A reads "Job 42 is due" at the same instant Scheduler B also reads "Job 42 is due." Both push it to the queue. Two workers pick it up and run it concurrently -> double execution, possibly double side effects (e.g., double email).

THE FIX – claim with an atomic status flip:

```
UPDATE jobs SET status='claimed', locked_by=$me, locked_at=now()  
WHERE id=42 AND status='pending'
```

This is a single atomic database operation. Only ONE caller's UPDATE can succeed in changing the row from 'pending' to 'claimed' – the database guarantees that. The loser sees "0 rows updated" and moves on.

For the **scheduler** process itself (the thing deciding "which jobs are due"), the simplest safe design is: **only one scheduler is active at a time** (a "leader"), elected using a coordination service like ZooKeeper or etcd (**leader election**: many candidate processes agree on exactly one "leader" who's allowed to act, with automatic handover if it dies). An alternative that avoids a single active scheduler is to **partition** the job space (e.g., by `job_id % N`) across multiple scheduler shards, each independently responsible for its slice – more throughput, slightly more operational complexity.

For **workers**, no leader election is needed – the atomic claim above is enough, because many workers claiming from the same queue is exactly what queues are designed for.

11. Replication

Three different things need replication here, so name each one:

Job DB: standard primary + replicas. Claims (writes) go to the primary; status-read traffic can go to replicas.

Queue: Kafka/SQS replicate messages across multiple brokers/nodes internally, so a "ready to run" job isn't lost if one broker dies before a worker reads it.

Scheduler: HA (high availability) via leader election – a standby scheduler is running at all times, watching the leader's health, ready to take over within seconds if the leader dies. Without this, a scheduler crash means NO new jobs get picked up until someone notices.

12. Multi-geo

Region: US-East	Region: EU-West
[Job DB (US)]	[Scheduler] [Queue] [Workers]
	...pinned to their region

Jobs are usually pinned to a region: a job submitted by a EU customer, whose payload references EU data, should run on EU workers, both for latency and for data-residency/compliance reasons (e.g., GDPR).

Cross-region job data generally should **not** be replicated everywhere the way an immutable URL mapping was in problem #1 — job payloads can contain sensitive customer data, and running a job in the "wrong" region can be a compliance violation, not just a performance issue. If a truly global scheduler is needed (e.g., "run this once, anywhere"), a lightweight global coordination layer picks *which* region owns a job, and that region's local DB/queue/workers take it from there — global coordination for *scheduling*, regional isolation for *execution and data*.

13. Failure handling

Worker dies mid-job	-> the job's "lease" (locked_at + a timeout, e.g. 5 min) expires -> another worker claims it and retries. THIS is exactly why jobs must be idempotent (section 4): the first worker might have half-finished the work before dying.
Job repeatedly fails	-> retry with EXPONENTIAL BACKOFF (wait longer between each retry: 1s, 2s, 4s, 8s...) so a failing dependency isn't hammered. After max_attempts, move the job to a DEAD LETTER QUEUE (DLQ) – a holding area for jobs that need a human to look at them, instead of retrying forever.
Scheduler (leader) dies	-> standby scheduler (section 11) is promoted via leader election; due jobs just get picked up a few seconds late.
Queue node/broker dies	-> queue's own replication (section 11) means the message isn't lost; consumers reconnect to a surviving broker.
DB primary dies	-> promote a replica; brief write pause on new submissions, but already-queued jobs keep running (they don't depend on the DB primary being up).
"Poison" job (always crashes the worker)	-> lease expires, retried, fails again, eventually hits max_attempts -> DLQ, instead of endlessly crashing workers.

Design principle: **assume every hop can fail, and make sure the job is never silently lost — worst case it runs late, or (rarely) twice, never zero times.**

14. Bottlenecks & scaling

- **Scheduler throughput** — if one scheduler must scan millions of "due" rows, shard the job table (e.g., by `job_id` or `job_type`) across multiple scheduler instances, each owning a slice, instead of one process scanning everything.
- **Queue throughput** — Kafka/SQS scale horizontally by adding partitions/shards; this is usually not the bottleneck.
- **Worker capacity** — this is the classic horizontal scale-out: add more worker machines, autoscaling on queue depth ("if the queue is backing up, add workers").
- **Hot job types** — a single job type suddenly submitted a million times (e.g., a mass notification) can starve other job types. Fix with **per-type queues or priority queues**, so a flood of low-priority jobs can't delay urgent ones.
- **DB write pressure from status updates** — every "claimed -> running -> succeeded" update is a DB write; at very high job rates, batch status updates or move completed-job history to a separate table/warehouse rather than updating the hot `jobs` table repeatedly.

15. Tradeoffs & what to say

"I split this into three concerns: scheduling (deciding a job is due), queuing (handing it off reliably), and execution (workers doing the work). I store job metadata in SQL because claiming a due job needs an atomic, transactional 'flip from pending to claimed' so two schedulers or workers never run the same job at once. Ready-to-run jobs move onto a message queue so workers just pull work without hammering the database. I commit to at-least-once execution — a job may run twice in rare failure cases — and make that safe by requiring every job to be idempotent, keyed by an idempotency key. Scheduler high-availability comes from leader election so there's always exactly one active scheduler, with a standby ready to take over. Failures at every hop are handled by lease expiry + retry with backoff, and a dead letter queue for jobs that never succeed, so nothing is silently lost."

Follow-ups an interviewer may probe - How do you support cron schedules efficiently at scale? (Precompute each schedule's *next* run time and store it, rather than parsing the cron string on every scheduler tick.) - What if a job needs to run on a *specific* worker (e.g., one with a GPU)? (Tag jobs with required capabilities; workers only pull jobs matching their own capabilities.) - How do you prevent thundering-herd at exactly 2am when thousands of daily jobs fire at once? (Jitter — add a small random offset to each job's scheduled time.) - How would you support job **priorities**? (Separate queues per priority, or a priority field the scheduler/queue respects when picking the next job.)

16. Video Streaming (YouTube / Netflix)

The problem in one line. Let creators upload videos, process them into many qualities behind the scenes, and let millions of viewers stream them smoothly — whether they're on fiber broadband or a shaky phone connection.

1. Requirements

Functional - Upload a video (large file, could be minutes to hours long). - The system processes the raw upload into several qualities automatically (so viewers don't all need the same fast internet). - A viewer can play the video, and it should play smoothly, switching quality as their connection changes — never a frozen spinner if we can avoid it. - Track view counts. - Out of scope: recommendations, comments, live streaming (that's a different design), monetization.

Non-functional - **Massive read (watch) traffic** — millions of people can watch the same popular video at once. Watches vastly outnumber uploads. - **Huge files** — a single video can be gigabytes. We cannot treat it like a small database row. - **Smooth playback** on all networks — buffering kills the experience. - **Durability** — once uploaded, a video must not be lost (people trust us with their content). - Upload can be slow/interrupted (mobile networks) — must be resumable.

2. Estimation

Assume 500 hours of video uploaded per minute (YouTube-scale, illustrative).
→ $500 * 60 = 30,000$ hours/hour of raw video coming in.
Assume average video = 10 minutes, raw size ≈ 1 GB (before compression).
uploads/day $\approx (500 * 60 * 24) / 10 = 72,000$ videos/day.
raw storage added/day $\approx 72,000 * 1 \text{ GB} = 72 \text{ TB/day}$ (before we even make the extra quality copies!).
After transcoding into ~ 5 qualities (240p..1080p/4K), each smaller than the raw master but still real size, storage roughly DOUBLES to TRIPLES per video.
→ object storage must be built for petabytes, growing daily. This alone tells us: raw video bytes CANNOT live in a normal database.
Watch QPS: a popular video can be watched by millions in a day; a single CDN edge server near a big city might serve tens of thousands of concurrent streams. This tells us: we need a CDN (Content Delivery Network — a global network of edge caches that serve files from near the user), not our own origin servers straining under every view.

3. API style — REST vs gRPC vs GraphQL

Choice: REST for metadata/upload control; plain HTTP GET (via CDN) for the actual video bytes.


```
POST /api/v1/uploads          -> start an upload, returns upload_id
PUT  /api/v1/uploads/{id}/chunk -> upload one chunk of the file (repeat)
POST /api/v1/uploads/{id}/complete -> mark upload done, kick off processing
GET  /api/v1/videos/{id}       -> metadata: title, status, manifest URL
GET  <cdn>/videos/{id}/hls/manifest.m3u8 -> the "menu" of qualities
GET  <cdn>/videos/{id}/hls/1080p/seg-042.ts -> one small video chunk
```

Why REST: upload and metadata are simple resource operations (create, read, update status) — REST's verbs map directly onto them, and any HTTP client (browser, mobile app) can call them with no special library. **Why not GraphQL:** there's no complex nested object graph to flexibly query here — a video's metadata is a small, fixed shape. **Why not gRPC for the public API:** browsers and video players speak plain HTTP natively; gRPC would add friction for zero benefit on this path. (Internally, the transcoding pipeline's worker-to-worker calls could use gRPC for speed, but that's an implementation detail, not the public contract.)

4. Idempotency — needed?

Yes, in two different places. - **Upload retries:** mobile uploads get interrupted constantly. We give the client an `upload_id` up front. If the connection drops mid-upload, the client resumes with the *same* `upload_id` and just re-sends the missing chunks — it does **not** start a brand-new upload. This is what makes "resumable upload" safe: retrying never creates a duplicate video. - **Transcoding jobs:** each transcode task is keyed by `(video_id, quality, segment_number)`. If a worker crashes halfway and the job is retried, it just re-produces the *same* output file at the *same* storage key — writing it again causes no harm (this property — repeating an action has the same effect as doing it once — is exactly what "idempotent" means).

5. Network protocol

HTTPS for everything, but split into two very different kinds of traffic: - **Control/metadata calls** (upload start, status, manifest fetch): normal HTTP/1.1 or HTTP/2 request/response. - **Actual video streaming:** also plain HTTP GET requests, but for small video *segments* (a segment = a few seconds of video saved as its own small file). This approach is called **HLS or DASH — HTTP-based adaptive streaming**. We do **not** invent a custom streaming protocol (like a raw TCP video socket) because: - Plain HTTP files can be **cached by any CDN** out of the box — CDNs are built to cache HTTP responses, not custom protocols. - HTTP passes through corporate firewalls and proxies with zero fuss. - If one segment's request fails, the player just re-requests that one small file — no need to re-establish a whole custom streaming session.

6. Database — SQL vs NoSQL

Choice: a normal relational (SQL) database for video *metadata*, and object storage (S3-like — a cheap, near-infinite key-based file store, NOT a database with queries) for the actual video bytes.

Why split like this: metadata (title, uploader, status, duration) is small, structured, and benefits from real queries — "find all videos by this uploader," "find videos still processing." That's exactly what a relational DB is good at. But a video file can be gigabytes; you would never want a multi-gigabyte blob sitting inside a database row — it would blow up backup size, replication traffic, and query performance for *everyone* sharing that database. **Object storage is built for exactly this:** durable, cheap, scales to

petabytes, and you fetch a file by its key (like a filename) — no joins, no transactions needed for a video segment.

View counts: don't write to the database on every single view (imagine a viral video getting 10,000 views/second — that many row updates would crush the DB). Instead, count views in a fast in-memory/stream layer and **flush aggregated totals** to the database every few seconds (see section 9 and 10 for exactly how).

7. Data model (detailed)

Table: videos (SQL)

```
-----
video_id      (PK)
title         (STRING)
uploader_id   (STRING)
status        (ENUM: UPLOADING | PROCESSING | READY | FAILED)
duration_sec  (INT)
created_at    (TIMESTAMP)
view_count    (INT)          <- updated by async aggregator, not per-view
-----
```

Table: uploads (SQL) -- tracks in-progress resumable uploads

```
-----
upload_id     (PK)
video_id      (FK, created up front so retries attach to it)
bytes_received (INT)          <- lets client ask "where do I resume from?"
status        (ENUM: IN_PROGRESS | COMPLETE)
-----
```

Manifest (stored in object storage as a small text file, NOT the SQL DB)

```
manifest.m3u8 for video_id=42:
  #QUALITY 240p -> list of segment URLs: seg-000.ts, seg-001.ts, ...
  #QUALITY 480p -> seg-000.ts, seg-001.ts, ...
  #QUALITY 1080p -> seg-000.ts, seg-001.ts, ...
```

Object storage layout (S3-like, key -> bytes):

```
/videos/42/raw/master.mp4          (original upload, kept as backup)
/videos/42/hls/240p/seg-000.ts ... seg-N.ts
/videos/42/hls/480p/seg-000.ts ... seg-N.ts
/videos/42/hls/1080p/seg-000.ts ... seg-N.ts
/videos/42/hls/manifest.m3u8
```

A **manifest** is simply a menu: it lists, for each quality, the URLs of all the small segments that make up the full video. The player downloads the manifest once, then downloads segments one at a time as the video plays.

8. Load balancer — L4 vs L7 vs API Gateway

```
UPLOAD / METADATA PATH (small, control traffic):
  Client -> [ L7 LB / API Gateway ] -> Upload Service
                                           -> Metadata Service
                                           (TLS termination, auth, routing by path, rate limits)

WATCH PATH (huge, video-byte traffic) -- bypasses our own LB entirely:
  Player -> [ CDN edge server, nearest to the user ] -> (cache miss?)
                                           -> Object Storage (origin)
```

Why L7/API Gateway for uploads and metadata: we need path-based routing (`/uploads/*` vs `/videos/*` go to different services), TLS termination, and auth/rate-limiting on the write path — all classic L7 features, the same reasons as in simpler systems. **Why the watch path skips our load balancer entirely:** video bytes are far too high-volume to push through our own servers at all — that's precisely the job a **CDN** exists to do. Think of the CDN as a giant, geographically-distributed load balancer *and* cache combined, purpose-built for exactly this kind of large-file, read-heavy traffic.

9. Caching + data flow (DB ↔ cache ↔ CDN)

There are two very different things being cached here: small metadata, and huge video bytes.

```
WATCH PATH (this is where almost all the traffic and cost is):
  Player asks for manifest.m3u8
    -> CDN edge near player: cache HIT? (very likely, video is popular)
        YES -> return cached manifest immediately
        NO  -> fetch from object storage (the "origin"), cache it, return
  Player asks for seg-042.ts (one quality's segment)
    -> same CDN cache-aside pattern: edge cache first, origin on miss
  Because CDN edges are close to users, most segment requests never
  touch our origin storage at all once a video is "hot" (popular).

METADATA PATH (small, changes rarely once video is READY):
  GET /videos/{id} -> check Redis cache -> hit? return
                                     -> miss? read SQL DB, cache it
```

Why this works so well: once a video finishes processing, its segments never change — they're **immutable**. Immutable data is the easiest thing in the world to cache aggressively and for a long time, because there's no invalidation problem (the cached copy can never go stale).

10. Concurrency

- **Transcoding is embarrassingly parallel.** A one-hour video can be split into many small segments, and each segment, at each target quality, can be transcoded (converted into another resolution/bitrate) by a *different* worker at the *same* time. We don't wait for one giant serial job — we fan out work across a pool of workers and fan back in once all pieces finish.
- **View counting under concurrency:** if we tried to `UPDATE videos SET view_count = view_count + 1` in the database on every single view, a viral video would create a huge number of workers all fighting to update the *same* row — a classic hot-row contention problem. Instead, view events go into a stream/queue; a separate aggregator counts them in memory (or in a fast counter store) per

short time window, then periodically **flushes** a single combined increment to the database. Far fewer, far cheaper writes.

- **Upload chunk ordering:** chunks of a resumable upload can, in principle, arrive out of order over an unreliable connection; the upload service tracks `bytes_received` and chunk offsets so it can reassemble the file correctly regardless of arrival order.

11. Replication — needed?

Yes, for both halves of the data. - **Object storage** (the video bytes) is replicated across multiple machines, and usually across multiple regions/data-centers, automatically by the storage system. Losing a creator's only copy of a video is unacceptable — replication is what makes "durable" a real guarantee rather than a hope. - **Metadata database** uses read replicas (extra copies that serve read traffic) since `GET /videos/{id}` is called far more often than videos are updated. Replica lag is basically harmless here — metadata like "title" or "status: READY" rarely changes after upload finishes, so a slightly stale replica is very unlikely to be wrong. - **The CDN itself** is inherently a giant replicated system — the same segment file sits cached on many edge servers around the world at once.

12. Multi-geo

Upload: user uploads to the NEAREST region's upload endpoint (fastest upload, avoids sending gigabytes across an ocean).
After processing, segments get pushed out to CDN edges worldwide (or pulled lazily to an edge the first time someone near it watches -- "pull CDN").

Watch: GeoDNS / CDN routing sends the player to the nearest edge server automatically. Object storage itself is also replicated cross-region so even a CDN cache miss doesn't require crossing the globe to the origin.

Benefit: a viewer in Tokyo streams from a Tokyo-area edge server, not from wherever the uploader happened to be — this is the entire reason CDNs exist for video.

13. Failure handling

One transcode job fails (e.g., worker crashes on segment 42, 1080p)
→ retry just THAT (video_id, quality, segment) job. Because jobs are idempotent (section 4), a retry is completely safe. No need to redo the whole video.

Upload interrupted (phone loses signal at 60% uploaded)
→ client resumes later with the same upload_id; server already knows bytes_received, so only the missing tail is sent.

CDN edge cache miss (video just went viral, edge hasn't cached it yet)
→ edge fetches from origin object storage once, caches it, all SUBSEQUENT nearby viewers get a cache hit. Just a one-time cost.

Origin object storage node fails
→ replication (section 11) means another copy answers; no video is lost.

Whole region down
→ CDN routes viewers to the next-nearest region/edge; cross-region replication of object storage means the video is still available.

Design principle: almost every failure here is scoped to one small piece (one segment, one chunk, one job) — never the whole video — *because* we broke the big problem into many small, independent, retryable pieces.

14. Bottlenecks & scaling

- **Watch traffic** (by far the biggest number) → absorbed almost entirely by the CDN; our own origin only serves the rare cache miss. This is the single most important scaling decision in the whole design.
- **Transcoding compute** → scales horizontally by adding more worker machines to the pool; because jobs are small and parallel (section 10), more workers directly means faster processing, with no coordination bottleneck.
- **Upload bandwidth** → resumable/chunked upload plus uploading to the nearest region spreads load and tolerates flaky connections.
- **View-count writes** → moved off the hot path entirely via async aggregation (section 10); the database never sees per-view traffic.
- **Storage growth** → object storage scales by simply adding more capacity under the hood; it's designed for this from day one, unlike a single database server with a fixed disk.

15. Tradeoffs & what to say

"The core insight is that video streaming is really two very different problems glued together: a small, structured metadata problem (SQL fits fine) and a huge-file streaming problem (object storage + CDN, not a database). I split storage that way rather than forcing everything into one system. Uploads are chunked and resumable, keyed by an upload_id, so retries are idempotent and never duplicate a video. After upload, a parallel transcoding pipeline produces multiple qualities split into small segments, described by a manifest — this is what lets adaptive bitrate streaming (HLS/DASH) work: the player just picks whichever quality fits its current bandwidth, segment by segment, over plain HTTP so CDNs can cache it. View counts are aggregated asynchronously instead of written per view,

because a hot row under millions of increments would fall over. Everything is replicated — storage across regions, metadata via read replicas, and the CDN itself is inherently a replicated, geo-distributed cache — so both durability and worldwide low-latency playback fall out naturally."

Follow-ups an interviewer may probe - How do you choose which qualities to generate for every video? (Cheaper to generate on-demand for rarely-watched videos, pre-generate all qualities for videos predicted to be popular.) - How does the player actually decide when to switch quality? (It measures recent segment download speed; if segments are arriving slower than they play, it requests the next segment at a lower quality — and vice versa.) - What about DRM/piracy protection? (Encrypt segments, issue short-lived decryption licenses per viewer — an extra layer on top of this design.) - How would live streaming change this? (No pre-existing file to transcode — segments must be produced and pushed to the CDN within seconds of being filmed; much tighter latency budget throughout.)

17. File Storage & Sync (Google Drive / Dropbox)

The problem in one line. Let a user upload files, keep a folder in sync across all of their devices, and share files with others — even when a device was offline for a while.

1. Requirements

Functional - Upload and download files. - **Sync**: if I edit a file on my laptop, my phone and desktop should see the change too. - Share a file or folder with another user (read or edit access). - Work with **large files** (videos, zip archives) without re-sending the whole file on every small edit. - Keep working with **offline edits** — a laptop closes its lid, edits a file on the plane, then reconnects and syncs.

Non-functional - Durability — a stored file must basically never be lost (users trust it like a backup). - **Bandwidth-efficient sync** — if I change one paragraph in a 500MB file, don't re-upload 500MB. - **Eventually consistent across devices** — a change shows up on other devices within seconds, not instantly (that's fine for this product). - Handle many devices per user, and many users sharing the same file.

Term check: "**sync**" just means "make sure every device ends up with the same latest copy of the file."

2. Estimation

```
Assume 500M users, avg 10 GB stored each.  
Total raw storage = 500M * 10 GB = 5 PB (before dedup/compression)  
Assume chunk size = 4 MB (a "chunk" is a fixed-size slice of a file – more on this below).  
A 100 MB file = ~25 chunks.  
Assume 50M active uploads/edits per day.  
writes/sec = 50M / 86400 ≈ 580 writes/sec (of "some chunk changed") → bursty, peak high  
Assume each user has 3 devices that need a "something changed" notification.  
500M users * 3 devices = 1.5B long-lived notification connections (sharded across servers)  
Metadata row per file: ~1 KB (name, path, version list, chunk hashes).  
500M users * ~10k files each = 5B files → 5B * 1KB = ~5 TB of metadata (fits a  
sharded DB easily – much smaller than the actual file bytes).
```

The big number is **raw bytes** (petabytes) → that goes to **object storage** (cheap, built for huge blobs).

The **metadata** (which files exist, their versions) is comparatively tiny → that goes in a **database** built for fast lookups. Keeping these two separate is the first big design decision, and we come back to it in section 6.

3. API style — REST vs gRPC vs GraphQL

Choice: REST for the API, plus a separate **WebSocket / long-poll channel** just for "something changed, come sync" notifications.

POST /api/v1/files	upload a new file
PUT /api/v1/files/{id}/chunks/{hash}	upload one changed chunk (resumable)
GET /api/v1/files/{id}/download	download (redirects to object storage / CDN)
POST /api/v1/files/{id}/share	share with another user
GET /api/v1/changes?since=<cursor>	"what changed since I last checked?"
WS /api/v1/notify	server pushes "file X changed" in real time

Why REST: uploads and downloads are simple resource operations — "give me this file," "here is a new chunk" — REST's plain nouns-and-verbs style fits well, and REST plays nicely with resumable/chunked HTTP uploads. **Why not GraphQL:** there's no complex nested query problem here (a client mostly wants "my file list" or "one file's contents"), so GraphQL's flexible-query power would be overkill. **Why add a WebSocket channel:** REST is request/response — the *client* has to ask. But sync feels instant only if the *server* can push "hey, something changed" the moment it happens. That's exactly what WebSocket (or a long-poll fallback for clients that can't hold a socket open) is for.

4. Idempotency — needed?

Yes, and it comes almost for free in one spot. - **Chunk upload is naturally idempotent** because we key each chunk by its **content hash** (explained in section 7). If a chunk with hash `abc123` already exists in storage, uploading it again is a no-op — "I already have that, thanks." A network retry that re-sends the same chunk twice does no harm. - **Committing a new file version** (the step that says "here is the new list of chunks for this file") should use a **client-generated request ID**. If the client's phone loses signal right after committing and retries, the server sees the same request ID and returns the same result instead of creating two versions. - Plain downloads are safe reads — nothing to make idempotent there.

5. Network protocol

HTTPS for uploads/downloads (standard request/response, and it can be resumed in pieces).

WebSocket (with long-poll as a fallback) for change notifications — the server needs to *push* to the client the instant a file changes, and a plain HTTP request can't do that; the client would have to poll constantly, wasting battery and bandwidth. TLS everywhere, since files are often private.

6. Database — SQL vs NoSQL

Choice: split storage by *shape* of the data — this is the core idea of the whole system.

METADATA (small, structured, needs quick lookups/updates)
 -> a database (SQL like Postgres, or a NoSQL wide-column store — either works, pick SQL if you want easy transactions across a few rows, e.g. "rename a file and update its folder listing together")

CHUNK BYTES (huge, unstructured blobs, write-once/read-many)
 -> object storage (like S3) — built to hold billions of large, immutable blobs cheaply and durably. You do NOT want multi-megabyte binary blobs living inside a database row; that makes the DB huge, slow to back up, and expensive to scale.

Why this split: a database is good at *small, structured, frequently-updated* records — "what version is file X on, what chunks does it have." Object storage is good at *large, rarely-changing blobs* — "give me

the 4MB of bytes for hash `abc123`." Using each tool for what it's good at is the single most important decision in this design.

7. Data model (detailed)

Key term: a **chunk** is a fixed-size slice of a file (e.g., 4MB). Splitting files into chunks, rather than treating a file as one giant blob, is what makes efficient sync possible — explained fully below.

A 12 MB file, split into 4 MB chunks:

-----+-----+-----+-----
Chunk 1 Chunk 2 Chunk 3
hash: h1a2b3 hash: h4d5e6 hash: h7f8g9
-----+-----+-----+-----

If the user edits only the text near the end of the file, only Chunk 3 changes → only h7f8g9's new bytes get uploaded. Chunks 1 and 2 are untouched, so we skip re-sending them. This is called **DELTA SYNC** – sync only the delta (the difference).

Table: File

id, name, owner_id, folder_path, current_version_id

Table: FileVersion

(one row per saved version of a file)

version_id, file_id, created_at, chunk_list (ordered list of chunk hashes)

Table: Chunk

(metadata only – bytes live in object storage)

hash (PK) → object_storage_key, size, ref_count

Table: Device / SyncState

(per user, per device)

device_id, user_id, last_synced_version, notify_channel

Deduplication: two different users (or the same user in two folders) can easily have the exact same chunk — e.g., they both saved the identical PDF, or the file barely changed so most chunks repeat. Because we key a chunk by its **content hash**, storing it once and having a `ref_count` (how many files point to it) means we **store the bytes only once**, no matter how many files reference that hash. This is called **deduplication** and it saves enormous amounts of storage space in practice.

Chunking + hashing on upload, step by step:

1. Client splits the file into 4MB chunks.
2. Client computes a hash (e.g., SHA-256) of each chunk.
3. Client asks the server: "do you already have hashes [h1, h2, h3]?"
4. Server replies: "I have h1 and h2, I need h3."
5. Client uploads only the bytes for h3.
6. Client commits a new FileVersion whose chunk_list is [h1, h2, h3].

Steps 3–5 are the heart of **delta sync**: we only ever move bytes for chunks the server doesn't already have.

8. Load balancer — L4 vs L7 vs API Gateway

```
Client (desktop app / phone app / browser)
|
[ L7 LB / API Gateway ] <- TLS termination, routes by path,
|                          auth, rate limiting
+--> Metadata Service    (file tree, versions, sharing – talks to the DB)
+--> Upload/Download    (chunk bytes – talks to object storage / CDN)
+--> Notification Service (WebSocket/long-poll fan-out)
```

Choice: L7 / API Gateway. We need to route very different kinds of traffic to different backends — small, frequent metadata calls vs. large binary chunk uploads vs. long-lived notification connections — and only an L7 balancer can read the URL path (or headers) to make that decision. **Why not plain L4:** L4 only sees IP/port, so it can't tell "this is a chunk upload" from "this is a metadata call." The gateway also handles TLS termination once at the edge and can enforce per-user rate limits so one runaway sync client can't hog capacity.

9. Caching + data flow (DB ↔ cache)

UPLOAD (write) path:

1. Client hashes its chunks, asks "which of these do you already have?"
2. Server checks a fast cache (or the Chunk table) for those hashes.
3. Client uploads only the missing chunk bytes → object storage.
4. Server writes a new FileVersion row (metadata DB) pointing at the full chunk_list (old unchanged chunks + newly uploaded ones).
5. Server pushes a "file changed" notification to the user's other devices.

SYNC (read) path, on another device:

1. Device gets (or polls for) a "file X changed to version V" notification.
2. Device asks metadata service: "give me version V's chunk_list."
3. Device compares to the chunk_list it already has locally.
4. Device downloads only the chunks it's missing (often via a CDN, since chunk bytes are immutable once written — perfect for caching).

What we cache: recently-used file metadata (in something like Redis) so opening a file doesn't always hit the DB; and chunk bytes behind a **CDN**, since a chunk, once written, never changes (its hash is its identity) — that makes it a perfect, permanently-cacheable object. This mirrors problem #1's redirect caching: **immutable data caches beautifully.**

10. Concurrency

Key problem: two devices edit the same file while both are offline, then both reconnect. Whose edit wins? Neither should silently disappear — that would be data loss, which this product must never do.

Simple, safe approach: CONFLICT COPY (what Dropbox actually does)

- Each device tracks "what version was I based on?" (like a version vector – a small marker of which version each device last saw).
- Device A syncs first: file goes from v5 → v6. Fine, no conflict.
- Device B reconnects, says "I'm creating v6 from my v5 base" – but the server is already at v6 from a DIFFERENT edit. That's a CONFLICT.
- Server keeps A's edit as the "real" file, and saves B's edit as a separate file: "myfile (conflicted copy, Device B, July 17).docx".
- Nothing is lost – the user manually merges later if needed.

Why not silently overwrite? Silently picking a winner risks throwing away real work the user cared about.
Why not lock the file during edits? Devices are offline for hours or days at a time — a lock would mean nobody else could edit for that whole time, which defeats the purpose of a syncing, multi-device product.

11. Replication — needed?

Yes, at both layers, for different reasons. - **Object storage** already replicates chunk bytes across multiple disks/servers/zones internally (that's what makes services like S3 durable — this is usually a given, not something you build yourself in an interview). - **Metadata DB** needs its own **replicas** (e.g., a primary plus 2 read replicas) so that (a) a single disk failure doesn't lose the file tree/version history, and (b) heavy read traffic (checking "what version is this on?") doesn't bottleneck a single machine. - Slight replica lag is acceptable for *reading* file lists, but the version-commit step (deciding whose edit is v6) should go through the primary to avoid two devices thinking they both "won."

12. Multi-geo

A user in Tokyo should upload/download from a Tokyo object storage region, not a US one → lower latency, and some regulations require data to stay in-region (data residency / compliance).

Metadata: replicate across regions too, but a file's "primary" region is usually wherever its owner is based; sharing across regions means the metadata service may need to fetch from another region occasionally (small latency hit, rare compared to normal use).

For a shared file between a US user and a Japan user, most designs pick one "home region" for that file and accept slightly higher latency for the other party, rather than trying to keep every file live in every region at once.

13. Failure handling

Upload interrupted mid-way (laptop closes lid)

-> resume later: client re-asks "which chunks do you still need from me?" and only sends those. No need to restart the whole file.

Two devices edit offline, both reconnect at once

-> conflict copy (section 10) – nothing lost, both versions kept.

Device offline for days, then reconnects

-> "catch-up sync": device asks "what changed since my last known version?" and pulls just the deltas since then, not the whole file.

Notification service is down

-> devices fall back to slow polling ("any changes since X?") every few minutes until push notifications come back – sync still works, just less instantly.

Object storage briefly unavailable in one region

-> object storage's own internal replication serves the read from another copy; metadata DB is untouched, so the user still sees their file listed (maybe with a "download unavailable" retry).

14. Bottlenecks & scaling

- **Total storage volume (petabytes)** → object storage scales horizontally by design; this is rarely the bottleneck in an interview discussion, but worth naming.
- **Metadata DB hot rows** (a very popular shared folder edited by many people) → shard the metadata DB by file/folder ID, and cache read-heavy metadata.
- **Notification fan-out** (one change needs to reach many devices, e.g., a shared team folder with 500 members) → don't push directly to 500 sockets one by one from a single service; fan the notification out through a pub/sub layer so it scales independently.
- **Popular chunk downloads** (a viral shared file) → CDN in front of object storage absorbs the read traffic, same idea as caching a popular URL redirect in problem #1.

15. Tradeoffs & what to say

"I split the file into fixed-size chunks and hash each one — that gives me delta sync (only upload what actually changed) and free deduplication (identical chunks stored once). Chunk bytes go to object storage because they're huge and immutable; metadata (file tree, versions, chunk lists) goes to a database because it's small and needs fast structured lookups — using the wrong tool for either would hurt. REST covers upload/download; a WebSocket/long-poll channel handles 'come sync now' push notifications, since polling alone would be slow and wasteful. For conflicting offline edits, I don't silently pick a winner — I keep both as a conflict copy, because losing user data is worse than a manual merge. Object storage replicates chunks automatically; the metadata DB gets its own replicas for durability and read scale."

Follow-ups an interviewer may probe - How small should a chunk be? (Smaller = finer-grained delta sync but more per-chunk overhead/metadata; 4MB is a common real-world compromise.) - How do you

avoid re-hashing a whole huge file on every tiny edit? (Rolling/content-defined chunking — recompute boundaries only near the edit, not the whole file — a deeper follow-up.) - What stops a device from downloading chunks it already has after reconnecting? (It sends its known chunk_list; server/CDN only sends the difference.) - How is sharing implemented? (A permissions table mapping file/folder → user → access level, checked by the metadata service before returning chunk locations.)

18. Payment System

The problem in one line. Move money correctly between people, merchants, and banks — charge a customer, pay out a merchant, record every movement, and never create or lose a single cent.

1. Requirements

Functional - Charge a customer's card/bank account for an order. - Pay out money to a merchant (send them their earnings). - Record every transaction in a **ledger** — a ledger is just an append-only record book of every money movement, like a very strict diary that you're never allowed to erase or edit. - Handle **refunds** (give money back for a completed charge). - Talk to external **payment providers** (Stripe, a bank's payment rail) that actually move the real money — we're the layer that decides *what* to move and *records* that it moved.

Non-functional - Correctness above all else. A social feed can drop a like; a payment system can never drop a dollar or double-charge a customer. This is the #1 constraint that shapes everything below. - **Strong consistency**, not eventual consistency — when we say a balance is \$50, it must really be \$50, not "probably \$50, will catch up soon." - Auditable: months later, we must be able to explain exactly why any balance is what it is. - Reasonably low latency (a checkout page shouldn't hang), but correctness wins if the two ever conflict.

2. Estimation

Assume 10M payments/day for a mid-size platform.
writes/sec = $10M / 86400 \approx 115/\text{sec}$ (peak $\sim 3-5x = \sim 500/\text{sec}$)
This is a LOW-QPS problem compared to, say, a news feed. The hard part isn't raw throughput — it's correctness under retries, crashes, and concurrent access to the SAME account.
Storage: every payment writes ≥ 2 ledger rows (a debit and a credit — explained in Section 7).
 $10M/\text{day} * 2 \text{ rows} * 200 \text{ bytes/row} * 365 \text{ days} * 5 \text{ yrs} = \sim 7.3 \text{ TB}$
over 5 years. Modest; a single well-indexed SQL cluster handles this comfortably. We keep everything — a ledger is never deleted.

3. API style — REST vs gRPC vs GraphQL

Choice: REST for the public-facing API that merchants/clients call; **gRPC** for the internal calls between our own services (payment service, ledger service, fraud service).

```
POST /v1/payments
  Headers: Idempotency-Key: 7e4b... (explained fully in Section 4)
  Body:    { "amount": 2000, "currency": "USD", "customer_id": "c_1" }
  ->      { "payment_id": "pay_1", "status": "SUCCESS" }

POST /v1/refunds    { "payment_id": "pay_1", "amount": 2000 }
```

Why REST: the outside world (a merchant's checkout page, a mobile app) just needs a simple "create this payment, get back a status" call — a resource-style POST is the natural fit, and REST is what every external client already knows how to integrate with. **Why not GraphQL:** there's no flexible-shape response problem here; the client always wants the same few fields back. **Why gRPC internally:** once a payment is accepted, our own Payment Service needs to call the Ledger Service and the Fraud Service quickly and with a typed contract — gRPC's binary format and strict schemas are a better fit for service-to-service calls than JSON-over-REST, and it's simply faster.

4. Idempotency — needed?

Yes. This is the single most important idea in this whole design. Idempotent means: "if you do it twice by accident, it only happens once." Here's why that matters so much for payments.

Imagine a customer clicks "Pay \$20" and the request reaches our server, we charge the card, but the network drops **before the response gets back** to the client. The client (or the client's app, automatically) sees no response and, very reasonably, **retries** the exact same request. Without protection, we'd charge the card a second time — the customer is now out \$40 for a \$20 order. This is the nightmare scenario a payment system must prevent.

The fix: an idempotency key. The client generates a random unique key (e.g., a UUID) once, *before* the first attempt, and sends it as a header on every retry of that same logical request. The server remembers every key it has already processed and, if it sees a repeat, returns the **original stored result** instead of charging again.

```
First attempt:
Client -> POST /v1/payments [Idempotency-Key: 7e4b...]
Server: never seen "7e4b..." before
    -> charge card -> write ledger entries -> store result
        keyed by "7e4b..." (status: SUCCESS, payment_id: pay_1)
    -> return { payment_id: pay_1, status: SUCCESS }
(network drops here before client sees the response)

Retry (same key, because it's the SAME logical request):
Client -> POST /v1/payments [Idempotency-Key: 7e4b...]
Server: "7e4b..." already exists!
    -> do NOT charge again
    -> just return the SAME stored result: pay_1, SUCCESS
```

How the server enforces this: the idempotency key is stored in a database column with a **unique constraint**. When a request comes in, the server tries to insert a new row with that key inside the same transaction that creates the payment. If another row with that key already exists, the unique constraint makes the insert fail immediately — a clean, race-safe signal that "this exact request was already handled," even if two retries arrive at almost the same instant. This is far safer than "check first, then act," which has a gap where two concurrent retries could both pass the check.

We also idempotency-key the **refund** call for the same reason — a network blip must never turn one refund into two.

5. Network protocol

HTTPS with TLS end to end — TLS is the encryption layer that keeps card numbers and amounts private in transit; there is no acceptable version of this system that isn't fully encrypted. Plain request/response is enough — there's no need for a WebSocket (a long-lived, two-way connection) since a payment is a single "do this, tell me the result" exchange, not a stream of events. When talking to an external provider like Stripe, we go over HTTPS to their API, and they call **us** back over HTTPS too (a "webhook" — covered in Section 13).

6. Database — SQL vs NoSQL

Choice: SQL (e.g., Postgres), with real ACID transactions. ACID is an acronym for four guarantees a database can give you: **Atomicity** (a multi-step change either fully happens or not at all — never half-done), **Consistency** (the data always obeys its rules, like "balances add up"), **Isolation** (concurrent transactions don't corrupt each other, covered in Section 10), and **Durability** (once we say "committed," it survives a crash).

Why SQL and not a NoSQL store: a NoSQL document/KV store's usual selling point is horizontal write scale with **eventual consistency** — writes are fast, but different readers might briefly see different, stale answers. For a news feed, a stale "like count" for a few seconds is invisible to users. For a bank balance, "briefly stale" means a customer could see \$100 and spend it twice before the system catches up — real money would go missing or get double-spent. We estimated only ~500 writes/sec at peak (Section 2) — nowhere near the scale that would force us into NoSQL's trade-offs. A single well-tuned Postgres cluster handles this easily, and in return we get **multi-row ACID transactions**, which is exactly what a double-entry ledger (Section 7) needs: writing two rows (a debit and a credit) that must **both** succeed or **both** fail, together, as one atomic unit.

7. Data model (detailed)

The centerpiece idea is the **double-entry ledger**. In double-entry bookkeeping, every single transaction writes **two** ledger rows that must balance: one **debit** (money leaving an account) and one **credit** (money arriving at an account), for the **same amount**. Nothing ever just "appears" or "disappears" — money only ever moves *from* one account *to* another, and the ledger proves it. The ledger is **append-only**: we never edit or delete a row; a mistake gets corrected by writing a new, opposite entry, so the full history is always intact.

Customer pays merchant \$20 for an order:

```
LedgerEntry 1:  account = customer_wallet    type = DEBIT    $20
LedgerEntry 2:  account = merchant_balance   type = CREDIT   $20
```

Both rows are written in ONE database transaction. Either both land, or neither does -> the books can never be "half updated."

An account's **balance is never stored as a raw number you just update** — it is *derived* by summing that account's ledger entries ($\text{sum}(\text{credits}) - \text{sum}(\text{debits})$, or the reverse sign convention, kept consistent). In practice we also keep a cached `balance` column on `Account` for fast reads, but the

ledger is the **source of truth**; the cached balance is rebuildable from it at any time by replaying the entries. If the two ever disagree, the ledger wins.

Table: Account

id	(PK)	
owner_id		customer or merchant
balance	(cached, derived from ledger)	
version	(INT)	for optimistic locking, Section 10

Table: LedgerEntry (append-only, NEVER updated/deleted)

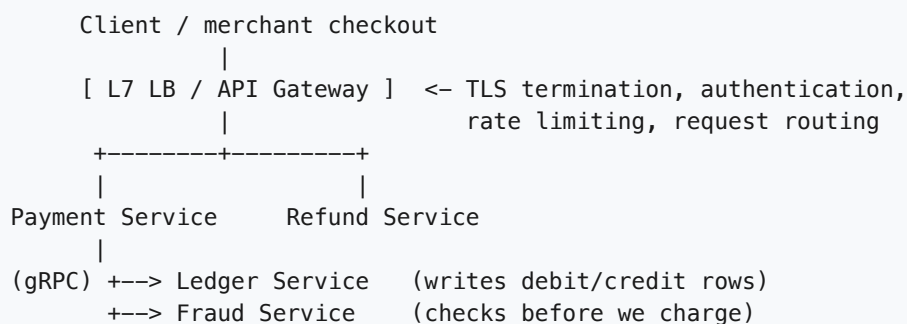
id	(PK)	
txn_id	(FK)	groups the debit+credit of one txn
account_id	(FK)	
type	(DEBIT CREDIT)	
amount	(INT, in cents)	never use floats for money
timestamp		

Table: Payment

id	(PK)	
idempotency_key	(UNIQUE)	Section 4 -- the dedup guard
status	(PENDING SUCCESS FAILED)	
amount		
provider_ref		Stripe/bank's own transaction id
created_at		

`Payment.status` starts at `PENDING` the instant we accept the request, moves to `SUCCESS` only after the provider confirms and the ledger rows are written, or `FAILED` if the provider declines. This status is what lets a retried request (Section 4) know what already happened.

8. Load balancer — L4 vs L7 vs API Gateway



Choice: L7 / API Gateway at the edge. Payments need **authentication** (who is this merchant/API key?) and **rate limiting** (stop someone from hammering the charge endpoint, which is both a cost and a fraud risk) — both are L7 (application-layer) features, since L4 only sees IP/port, not request headers or paths. The gateway is also the natural place to enforce "every request must carry an Idempotency-Key" before it even reaches the Payment Service.

9. Caching + data flow (DB ↔ cache)

We deliberately do NOT cache the authoritative balance for a debit decision. This is worth explaining clearly, because it's the opposite of most other problems in this guide.

Say a cache says "Account X has \$50" but the real, current database value is \$10 (because another debit landed a second ago and the cache hasn't caught up yet). If we approve a new \$30 debit based on the **stale cached \$50**, the account goes to **-\$20** — money we don't have has effectively been spent. A stale cache is *fine* for a redirect URL or a like count; it is **not fine** for "can this account afford this charge right now?" So: the money *decision* always reads the real database balance, inside the transaction that will also make the change (see Section 10's row lock).

Where caching *is* fine: things that are read often but never used to make a money decision — e.g., a customer's **payment history list** for a "past transactions" screen, or provider metadata that rarely changes. Those can be cached normally with a short TTL, because being a few seconds stale there costs nothing.

Money decision (debit an account):
ALWAYS read live from Postgres,
inside the same transaction that
will write the new balance.

Read-only display page:
OK to serve from a cache
(e.g., "your last 20
transactions" list).

10. Concurrency

Here is the classic race this design must prevent: **two debits hit the same account at nearly the same time**, and both read the balance *before* either one writes.

Account balance = \$10. Two debits of \$8 arrive concurrently.

Thread A: read balance (\$10) → OK, \$10 ≥ \$8 → about to debit

Thread B: read balance (\$10) → OK, \$10 ≥ \$8 → about to debit

Thread A: writes new balance = \$2

Thread B: writes new balance = \$2 (overwrites A's update!)

Result: balance is \$2, but \$16 left the account → should be -\$6.

The account drained below zero and we LOST TRACK of \$8.

The fix: `SELECT ... FOR UPDATE` (a row lock). This tells the database "lock this account's row the instant I read it — no other transaction may read-for-update it until I'm done." Thread B's read then simply **waits** until Thread A's transaction commits (or rolls back), so B sees the *true, updated* balance before deciding whether it can proceed.

Thread A: `BEGIN; SELECT balance FOR UPDATE WHERE id = X; (locked)`
→ \$10, OK → debit → balance = \$2 → `COMMIT` (unlocks)

Thread B: `BEGIN; SELECT balance FOR UPDATE WHERE id = X; (WAITS`
for A's lock to release)
→ now sees \$2 → \$2 < \$8 → `REJECT`, insufficient funds

An alternative is **optimistic locking**: read the `version` column (Section 7) along with the balance, and when writing, do `UPDATE ... WHERE id = X AND version = <the version you read>`. If another

transaction changed the row first, the version no longer matches, zero rows update, and we know to retry. This avoids holding a lock while doing other work, at the cost of having to retry on conflict. `SELECT FOR UPDATE` is simpler to reason about and is the usual default for money; optimistic locking suits cases with light contention and a desire for higher throughput.

11. Replication — needed?

Yes — but with a strong requirement: synchronous replication for the primary money database.

Synchronous means the primary **waits** for at least one replica to confirm it has the data too, *before* telling the client "committed." This matters because if the primary crashes one millisecond after committing a payment, we cannot afford for that committed transaction to simply vanish because it only ever existed on the now-dead primary. With synchronous replication, a replica already has it, and can be promoted to take over with no data loss.

```
Primary (Postgres)
| write ledger rows
| WAIT for replica ack before confirming to client (sync)
v
Replica (has the same committed data -- safe to promote if
        primary dies)

Additional ASYNC read replicas (lag OK) serve:
- reporting / analytics
- "view your transaction history" pages
  (never used for the live money decision in Section 9/10)
```

The trade-off: synchronous replication adds a little latency to every write (we wait for the network round-trip to the replica), but that's the right price to pay — losing a committed payment is unacceptable; a few extra milliseconds on checkout is not.

12. Multi-geo

Money has two extra multi-geo complications that most systems don't: **data locality laws** and **compliance**. Many countries require that certain financial data about their citizens stay physically within that country (data residency), and card data specifically falls under **PCI DSS** (Payment Card Industry Data Security Standard — a strict security/compliance rule set for anyone handling card numbers).

```
Region: EU          Region: US
Regional ledger (EUR)  Regional ledger (USD)
EU customer accounts  US customer accounts
PCI-scoped card data  PCI-scoped card data
stays in EU           stays in US

      \      /
       \    / cross-border payment /
        \  / (a distinct, harder /
         \ / flow -- currency <--'
          \  conversion, extra
           \  compliance checks)
```

Approach: keep a **separate regional ledger** per major region rather than one global ledger, so each region's data stays where regulation says it must, and each region's payments mostly stay fast and local.

A payment that crosses regions (e.g., a US customer paying an EU merchant) is handled as its own, more careful flow — it may need currency conversion, extra fraud/compliance checks, and often goes through a dedicated cross-border provider rather than our normal fast path. Say plainly in an interview: cross-border payments are meaningfully harder, and a good design calls that out rather than hand-waving it away.

13. Failure handling

Provider timeout — did the charge succeed or not? This is the scariest failure mode: we call Stripe/a bank, and the connection times out before we get a response. The charge might have gone through on their side, or it might not have — we genuinely don't know yet.

```
We call provider -> ... timeout, no response ...
We do NOT retry blindly (that risks a double charge if it DID
succeed on their end).
Instead:
1. Mark Payment as PENDING (already was).
2. Call the provider's "check status of this idempotency key /
reference" endpoint to find out what actually happened.
3. Once we know (SUCCESS or FAILED), update our Payment status
and write ledger entries accordingly — exactly once.
If OUR retry of the original charge request reaches the provider,
the idempotency key we sent THEM also protects us there — most
providers support the same idempotency-key pattern we do.
```

Webhooks — the provider tells us asynchronously. Providers also push events to us (e.g., "payment pay_1 finally succeeded") via a **webhook** — a callback HTTP request the provider makes to a URL we gave them. Webhooks can arrive **late, out of order, or more than once** (the provider retries webhooks too, for the same reasons we retry calls). We handle this exactly like Section 4: the webhook includes the provider's own reference id, we check "have I already processed an event for this reference," and if so we simply acknowledge it and do nothing more.

Multi-step payments — the SAGA pattern. Some payments aren't one atomic step; they're a sequence, e.g., **reserve funds -> charge -> confirm with merchant**. If step 3 fails after steps 1-2 succeeded, we can't just "roll back" like a single database transaction (the money has already moved at the provider). Instead we run a **compensating action** — a deliberate reverse step, e.g., "release the reservation" or "issue a refund" — to undo the effect of the steps that did succeed.

```
SAGA: Reserve -> Charge -> Confirm
                        |
                        fails here
                        |
run compensating actions in reverse:
undo Charge (refund) -> undo Reserve (release hold)
```

Every step and every compensating action is itself protected by an idempotency key, so retrying a saga step (or its compensation) never double-applies it. This combination — sagas for the sequence, idempotency for each step — is how we get an **exactly-once effect** even though the underlying network calls can and will be retried.

14. Bottlenecks & scaling

- **Write throughput on the ledger table** — since every payment is at least 2 rows in one transaction, and we computed only ~500/sec at peak (Section 2), a single primary handles this for a long time. If it ever becomes the bottleneck, **shard by account/customer** so unrelated accounts' transactions don't contend with each other.
- **Row-lock contention (Section 10)** on a very "hot" account (e.g., one huge merchant receiving thousands of payments a minute) — mitigate by batching credits to that account periodically instead of one immediate write per incoming payment, or by splitting the merchant's balance across sub-accounts that get reconciled together.
- **Read load for history/reporting** — push entirely to the async replicas (Section 11), never the primary, so reporting queries can never slow down real money movement.
- **Provider calls are the slowest step** (an external network hop to Stripe/a bank) — keep them off any path that holds a database lock, and use the PENDING status plus webhooks/idempotency so a slow provider never blocks other customers.

15. Tradeoffs & what to say

"Correctness dominates every decision here, so I pick a SQL database with real ACID transactions, not a NoSQL store — I need multi-row atomic writes for the double-entry ledger, and I cannot tolerate eventual consistency on a balance. Every request carries a client-generated idempotency key stored under a unique constraint, so a network retry can never double-charge — that's the single most important guarantee in the system. Every transaction writes a balanced debit and credit to an append-only ledger; balances are derived from it, never trusted as a raw mutable number. Concurrent debits on one account are serialized with `SELECT FOR UPDATE` so we can never overdraw by racing two reads. Replication to the money database is synchronous — I will not risk losing a committed transaction for a few milliseconds of latency. Multi-step flows use the saga pattern with compensating actions, and provider timeouts are resolved by checking status rather than blindly retrying. Every one of these choices trades a little speed or simplicity for the one property that actually matters: money is never created, never lost, and always explainable."

Follow-ups an interviewer may probe - How do you reconcile our ledger against the provider's records? (A periodic batch job compares our Payment/ledger totals against the provider's statement, flags mismatches for manual review — this is standard practice, not a sign something is broken.) - What if the idempotency key store itself grows huge? (Keys only need to be kept long enough to cover realistic retry windows, e.g., 24-48 hours, then archived/purged.) - How do you prevent fraud, not just double-charges? (Separate concern — a Fraud Service scores the request before we charge, sitting alongside, not inside, the ledger logic.) - Floats vs integers for money? (Always integers — smallest currency unit, e.g., cents — floats introduce rounding errors that are unacceptable for money.)

19. Distributed Message Queue (Kafka-style)

The problem in one line. Let many **producers** (senders) publish messages to a **topic** (a named stream), and many **consumers** (readers) reliably read those messages — durably, in order, at very high throughput, and even replay them later.

1. Requirements

Functional - A **producer** publishes messages to a **topic** — think of a topic as a named channel, like "orders" or "page-clicks". - A **consumer** subscribes to a topic and reads the messages, usually in the order they arrived. - Messages must be **durable** — once the system says "got it," a crash must not lose the message. - The system must handle **very high throughput** — millions of messages per second across many topics. - Consumers should be able to **replay** old messages (e.g., "re-read everything from yesterday") within some retention window, not just see brand-new ones.

Non-functional - **High write throughput** — this is the dominant concern; the design centers on making writes (appends) as fast as physically possible. - **Durability** — a message, once acknowledged, survives machine crashes. - **Ordering** — guaranteed only within a single **partition** (one ordered slice of a topic, explained in Section 7) — not across an entire topic. - **Horizontal scalability** — adding more machines should let the system handle more topics, more partitions, and more consumers, roughly linearly. - **Decoupling** — producers and consumers don't call each other directly; they don't even need to be online at the same time. The queue sits in between and buffers everything.

2. Estimation

```
Assume a mid-size company: 1M messages/sec written across all topics.  
average message size ~= 1 KB  
write throughput      ~= 1M * 1 KB = ~1 GB/sec of raw data  
Retention: keep 7 days of data for replay.  
storage = 1 GB/sec * 86400 sec/day * 7 days ~= ~600 TB  
-> way too much for one machine's disk; must be SHARDED across  
    many broker machines (a "broker" = one server in the cluster  
        that stores and serves partitions).  
Read side: often MANY consumer groups read the same topic  
(e.g., one group for real-time alerts, another for analytics,  
another for archiving) -> reads can be several times write volume,  
but reads are cheap sequential scans (see Section 6), so this is  
far less scary than it sounds.
```

This one calculation drives the whole design: **too much data for one disk, too much throughput for one machine** — so a topic must be split up and spread across many machines.

3. API style — REST vs gRPC vs GraphQL

Choice: a custom binary protocol over long-lived TCP connections (this is exactly what Kafka does), not REST and not classic gRPC-over-HTTP/2 either — though a gRPC-style generated-stub client is a reasonable modern variant of the same idea.

```
Producer client: connect once -> send batches of messages continuously
Consumer client: connect once -> poll() in a loop, get back a batch
                  of messages each time
```

Why not REST: REST means "open an HTTP connection, send one request, get one response," which has real per-call overhead (headers, connection setup, parsing). At millions of messages per second, paying that overhead per message is far too slow. **Why a custom protocol:** the client library can **batch** many messages into one network write, use a compact binary format instead of verbose JSON, and reuse one long-lived TCP connection for thousands of messages — all of which REST makes awkward. This is a case where "use REST for everything" is the wrong default: the hot path here is throughput, not developer convenience, so we trade a nicer API for a much faster wire format.

4. Idempotency — needed?

Yes, and this is a central design point for a queue — not a side note.

- **Producer side:** if a producer sends a message and the network blips before it gets an acknowledgment, does it retry? If it just resends blindly, the broker might end up storing the same message twice. The fix is **producer idempotence**: each producer gets a unique **producer ID**, and tags every message with a **sequence number** (1, 2, 3, ...). The broker remembers the last sequence number it accepted from that producer ID and simply **drops a duplicate** if it sees the same sequence number again. This turns "retry blindly" into "retry safely."
- **Consumer side:** the queue's default delivery guarantee is **at-least-once** (explained next in this section) — meaning a consumer might see the same message twice (e.g., it processed a message but crashed right before recording that fact). So **consumers must be written to be idempotent** too — e.g., "applying this payment twice has the same effect as applying it once," often by keying on a message's unique ID and checking "have I already handled this ID?" before acting.

Delivery semantics, explained simply:

```
At-most-once: send it, don't worry if it's lost. Fast, but a crash
               can silently drop a message. Rarely what you want.
At-least-once: (the common default) resend if unsure it arrived.
               Never silently loses a message, but the SAME message
               might arrive twice -> consumer must be idempotent.
Exactly-once:  the hard one. Needs producer idempotence (above) PLUS
               the consumer's "read a message" and "write my result"
               to happen as one atomic unit (a transaction spanning
               both) -> more overhead, used only when duplicates are
               truly unacceptable (e.g., money movement).
```


5. Network protocol

Long-lived TCP connections, not per-message connections. A producer or consumer opens one TCP connection to a broker and keeps it open, streaming many messages over it — opening a fresh connection per message would be enormously wasteful (TCP handshakes are slow relative to sending a 1 KB message).

Batching is the single biggest throughput lever. Instead of sending each message the instant it's produced, the client library holds messages for a few milliseconds (or until a batch reaches some size) and sends them together as one bigger network write. This trades a tiny bit of latency (a few ms of buffering) for a huge gain in throughput, because it amortizes per-request overhead (network round trip, broker bookkeeping) across many messages instead of paying it per message.

6. Database — why a log beats a traditional DB here

Choice: the queue's own storage IS the "database" — an append-only commit log on disk. There's no separate MySQL/Postgres/DynamoDB underneath; the broker writes messages straight to files on disk, in the exact order they arrive, and never edits or rewrites old ones.

Why a plain log wins over a general-purpose database: - A traditional DB is built to support arbitrary reads, updates, and deletes anywhere in the data — which means random-access disk I/O (jump to row X, update it in place) and lots of bookkeeping (indexes, transaction logs, locks) to make that safe. All of that is overhead we don't need here. - A message queue only ever does one kind of write — **append to the end** — and mostly one kind of read — **read forward from some position**. Both of those map to **sequential disk I/O**, which is dramatically faster than random I/O, even on spinning disks, because the disk head never has to jump around. - No updates, no deletes (until retention expires and we drop old files), no random access → we can skip almost all of a database's overhead and just write raw bytes to the end of a file as fast as the disk allows.

```
Traditional DB write: find the row -> lock it -> update in place
                     (random I/O, lots of bookkeeping)
Log write (this design): just append bytes to the end of a file
                        (sequential I/O, minimal bookkeeping)
                        -> much higher raw throughput
```

7. Data model (detailed)

The centerpiece idea: Topic → Partitions → ordered messages.

A **topic** (e.g., "orders") is too big and too busy for one machine, so it's split into several **partitions**. A partition is simply **one ordered, append-only log** — a single file (or set of files) that messages get appended to, one after another. Each message landing in a partition gets an **offset**: its position in that log, starting at 0, 1, 2, ... — think of it as a line number in a very long file.

Topic: "orders" (split into 3 partitions, spread across brokers)

```
Partition 0: [ msg@0 | msg@1 | msg@2 | msg@3 | ... ] (append-only)
Partition 1: [ msg@0 | msg@1 | msg@2 | ...           ]
Partition 2: [ msg@0 | msg@1 | ...                   ]
```

Each message stored:

offset	(INT)	position within its partition
key	(BYTES, optional)	used to pick the partition
value	(BYTES)	the actual message payload
timestamp	(LONG)	when it was produced

How a producer picks a partition: if the message has a **key** (e.g., a customer ID), the producer hashes the key to always send that key's messages to the *same* partition — this is how you get ordering "per customer." With no key, the producer just spreads messages round-robin across partitions for even load.

This is exactly why ordering is only guaranteed within one partition — messages in different partitions are written by different, unsynchronized appenders, so there's no single global order across the whole topic.

Consumer offsets — how far each consumer has read — are themselves stored durably (often in a special internal topic), so a consumer that restarts knows exactly where to resume, instead of starting over or guessing.

8. Load balancer — why smart clients replace a classic LB here

Choice: no classic load balancer in front of the brokers — clients route themselves using metadata.

```
Client
|
1. Ask any broker (or a metadata service): "who is the LEADER
   for partition 2 of topic 'orders'?"
2. Broker replies: "broker #5"
3. Client connects DIRECTLY to broker #5 for that partition
```

Why not a normal L4/L7 load balancer: an LB is great when any backend instance can handle any request. Here that's false — each partition's data physically **lives** on one specific leader broker (Section 11), so a request for partition 2 *must* reach the broker that actually holds partition 2's log. Routing that through a generic load balancer would add a hop and still require the LB to somehow know per-partition ownership. Instead, the **client library itself is "smart"**: it fetches a small map of "which broker leads which partition" (this map is called **metadata**) and connects straight to the right broker, skipping a middleman entirely.

9. Caching + data flow (producer → replicate → consumer)

There's no separate cache layer to design — the trick here is that **the operating system's page cache does the job for free.**

PRODUCE:

Producer -> batches messages -> sends to partition leader broker
-> leader appends to its log file on disk
-> leader replicates to follower brokers (Section 11)
-> once enough replicas have it, leader ACKs the producer

CONSUME:

Consumer -> poll() -> asks leader broker "give me messages after offset X" -> broker reads forward from that offset -> returns a batch of messages -> consumer processes them, then records "I'm now at offset X+N" (its new committed offset)

Why reads are fast without a dedicated cache: consumers usually read *recently written* data (they're mostly caught up, reading close to the end of the log). Recently written data is still sitting in the operating system's **page cache** — RAM the OS uses to cache disk pages automatically, with no extra code required. So a "read from disk" often turns out to be a fast RAM read in practice, purely because of *how* the data is accessed (sequential, recent) — one more benefit of the append-only log design from Section 6.

10. Concurrency

- **One consumer per partition, per consumer group.** A **consumer group** is a set of consumers cooperating to read one topic — think of it as "one team of workers splitting up the reading." The queue assigns each partition to exactly one consumer within a group, so two consumers in the same group never fight over the same partition. This is how work gets divided cleanly: 6 partitions and 3 consumers in a group means each consumer handles 2 partitions, with **zero contention** — no locks needed, because there's simply nothing shared.
- **Ordering is per-partition only.** Because exactly one consumer reads a given partition at a time, and that partition's messages were appended in a strict order, that consumer sees them in order. Across *different* partitions there's no such guarantee — two consumers reading two different partitions run fully independently.
- **Multiple consumer groups can read the same topic simultaneously**, completely independently of each other (e.g., one group for real-time alerting, another for nightly analytics) — they each track their own offsets, so one group's pace never affects another.

11. Replication — leader/follower per partition

Yes — this is how the log survives a machine dying, not just how it scales reads.

Partition 0 (replication factor = 3):

Broker A: [LEADER] <- producers & consumers talk to this one
Broker B: [follower] <- continuously copies A's log
Broker C: [follower] <- continuously copies A's log

- Every partition has one **leader** (handles all reads and writes for that partition) and some number of **follower** replicas that continuously copy the leader's log.
- The set of replicas that are fully caught up with the leader is called the **ISR** (in-sync replicas) — this matters because only ISR members are safe candidates to become the new leader if the current

leader dies (an out-of-date follower promoted to leader could serve stale data or silently drop messages the old leader already acknowledged).

- **acks setting controls the durability/latency tradeoff:** `acks=0` : producer doesn't wait for any confirmation. Fastest, but a message can vanish if the leader crashes before writing it. `acks=1` : producer waits for the LEADER to write it. Safe against most failures, but if the leader dies before followers copy it, that message is lost. `acks=all` : producer waits until ALL in-sync replicas have the message. Slowest (extra network round trips), but a message survives even if the leader dies immediately after acking. This is a direct dial between speed and safety — a payments topic should probably use `acks=all`; a low-stakes clickstream topic might use `acks=1` for speed.
- **Retention:** even after every consumer has read a message, the broker keeps it on disk for a configured retention period (e.g., 7 days) rather than deleting it immediately — this is what makes **replay** possible: a consumer can rewind to an earlier offset and re-read history, or a brand-new consumer can start from the very beginning.

12. Multi-geo

Region US-East

Region EU-West

Cluster (topic "orders")
 leaders + followers
 all within this region

Cluster (mirrored copy
 of topic "orders")

A cross-region "mirroring" job continuously reads from
the source cluster and re-produces into the destination
cluster's matching topic.

Each **region runs its own independent cluster** (partitions and their leader/follower replicas normally all live within one region — replicating every write synchronously across regions would add too much latency to every produce call). To make data available in another region, a separate **mirroring** process (a consumer in region A that's also a producer into region B) copies topics across, on its own schedule, decoupled from the live write path. This keeps regional writes fast while still letting a second region catch up asynchronously — useful for disaster recovery or serving regional analytics locally.

13. Failure handling

- A broker (node) holding a partition LEADER dies
 - > the cluster detects the failure (via a coordination service)
 - > one of the in-sync follower replicas (Section 11) is promoted to be the new leader
 - > producers/consumers refresh their metadata (Section 8) and simply reconnect to the new leader; no data is lost because the new leader was already caught up (it was in the ISR)
- A consumer in a group dies
 - > the group notices (it stops sending heartbeats)
 - > the group REBALANCES: its partitions are reassigned to the remaining live consumers in that group
 - > each reassigned partition resumes from its last COMMITTED offset, so no message is skipped -- though a message processed-but-not-yet-committed may be redelivered (this is exactly the at-least-once behavior from Section 4)
- A whole region goes down
 - > other regions are unaffected (independent clusters); the mirrored copy (Section 12) can be promoted to primary for that topic if needed, accepting some replication lag as loss

Design principle: **a partition's replicas make individual broker failure a non-event, and consumer group rebalancing makes individual consumer failure a non-event** — the two failure-recovery mechanisms are almost entirely separate concerns, one on the storage side and one on the reading side.

14. Bottlenecks & scaling

- **Write throughput** → add more partitions (spreads appends across more brokers) and add more broker machines; sequential-disk-write design (Section 6) means each individual partition is already about as fast as one disk allows.
- **Read throughput / many consumer groups** → cheap, because each group reads independently and reads are sequential scans that mostly hit the OS page cache (Section 9) — adding another consumer group reading the same topic barely affects existing ones.
- **A single "hot" partition** (e.g., one key gets way more traffic than others) → the partition it maps to becomes a bottleneck no matter how many other partitions exist; fix by choosing a better partition key (more even spread) or splitting that key's traffic.
- **Too few partitions vs. too many consumers** → extra consumers beyond the partition count sit idle (one consumer per partition per group, Section 10) — partition count sets the ceiling on how much a single group can parallelize.
- **Storage growth** → bounded by the retention period (Section 11); shortening retention or moving very old data to cheaper long-term storage controls disk usage.

15. Tradeoffs & what to say

"A topic is split into partitions, each an append-only log with sequential disk writes — that's what makes this so much faster than a general-purpose database for this workload. Producers batch messages over long-lived TCP connections instead of REST, because at

this throughput per-message overhead adds up fast. Consumer groups divide partitions one-to-one so there's no contention, and ordering is guaranteed per-partition, not topic-wide. Durability comes from leader/follower replication with a tunable `acks` setting — trading latency for safety — and clients route directly to the right broker using metadata instead of going through a classic load balancer, since data placement is fixed per partition. At-least-once delivery is the practical default, so consumers must be idempotent; true exactly-once needs producer idempotence plus transactional reads-and-writes. Failure is handled by promoting an in-sync follower on broker death, and rebalancing partitions across the group on consumer death — both recover without losing committed data."

Follow-ups an interviewer may probe - How do you choose the number of partitions? (Balance parallelism/throughput against per-partition overhead and rebalancing cost — more partitions isn't free.) - What happens if a consumer is just slow, not dead? (It keeps its partitions but lags behind; monitor consumer lag — the gap between the latest offset and the consumer's committed offset — as a key health metric.) - How is exactly-once actually implemented? (Producer idempotence, Section 4, plus wrapping "read input, write output, commit offset" in one atomic transaction.) - Why not just use a traditional database table as a queue? (Random I/O and lock/index overhead per row, versus pure sequential appends — Section 6 — at far higher throughput.)

20. Search Engine / Log Search

The problem in one line. Given millions (or billions) of documents — web pages, product listings, or log lines — let a user type a free-text query and get back the most relevant matching documents, fast, with filters and ranking.

1. Requirements

Functional - `index(document)` — add or update a document (id + text fields + metadata) so it becomes searchable. - `search(query, filters, page)` — given free text (e.g. "error database timeout"), return the top matching documents, ranked by **relevance** (how well they match), not just any match. Support filters (e.g. `service = "checkout"`, `timestamp > yesterday`) and pagination. - Support phrase/keyword search at a minimum; bonus points for typo-tolerance later.

Non-functional - **Low query latency** — a search box or a log dashboard should respond in well under a second, even across billions of documents. - **High write volume for logs** — a log-search system (think "search my application logs") can ingest millions of new lines per second; a web/product search system writes far less often but reads constantly. - **Scale out, not up** — the dataset is too big for one machine's disk or RAM, so the index itself must be **split across many machines**. - Slightly **stale** results (a newly indexed document takes a second or two to become searchable) are acceptable — this is called **near-real-time**, not real-time.

2. Estimation

Assume a log-search use case: 10M log lines/sec across a company.
average line ~300 bytes -> raw ingest = 3 GB/sec (!)
Over a day: 3 GB/sec * 86400 sec ~= 260 TB/day
-> far too big for one machine -> MUST shard (split) the index across many nodes, and probably only keep hot data (e.g. last 7-30 days) in the fast search tier; older data goes to cold storage.
Query volume: much lower than writes for logs (engineers searching, not every request). Maybe 100s-1000s of searches/sec company-wide.
Index size vs raw size: an inverted index (explained in Section 7) is usually 10-30% of the raw text size once compressed -> still tens of TB, confirming we need many shards, not one big disk.

3. API style — REST vs gRPC vs GraphQL

Choice: REST for the public/query-facing API; a **bulk write API** (still REST, but batched) for high-volume ingestion.

```
GET /search?q=error+timeout&service=checkout&page=2
-> { "hits": [ {docId, score, snippet}, ... ], "total": 5321 }

POST /_bulk      (many documents in one request body)
[{ "index": { "id": "doc123" } },
 { "text": "...", "service": "checkout", "ts": ... },
 ... many more pairs ...]
```

Why REST: the query is a simple "give me text + filters, get back a ranked list" — exactly what a `GET` with query parameters expresses well. **Why bulk for writes:** if we called a single-document write API millions of times per second, the per-request network and parsing overhead alone would be a bottleneck. Batching many documents into one HTTP call amortizes that cost. **Why not GraphQL:** there's no nested-object over-fetching problem to solve here. **Why not gRPC for the public API:** REST/JSON is what search clients and dashboards expect; gRPC is fine *internally* between the coordinator and shard nodes (Section 8) where every millisecond of serialization overhead matters more.

4. Idempotency — needed?

Yes, and it's easy here. - **Indexing a document is an upsert keyed by document id** — "insert this document, or if a document with this id already exists, replace it." Sending the *same* document twice just overwrites it with itself — no duplicate, no error. That's what idempotent means: repeating the operation causes no extra side effect. - **Reads (search) are naturally idempotent** — asking the same question twice just returns the same answer (until the data changes underneath it), with no side effects at all. - The one thing to watch: if a client retries a **bulk** write after a partial failure, some documents in that batch may already be indexed. Because indexing is upsert-by-id, resending the whole batch is still safe — the already-indexed documents simply get overwritten with identical data.

5. Network protocol

HTTPS/HTTP2 for both the query API and bulk ingestion. HTTP/2 lets many small search requests or many bulk-write chunks share connections efficiently (multiplexing — sending several requests over one connection instead of opening a new one each time), which matters at high QPS. No need for WebSocket — there's no ongoing server-push relationship, just request/response, even for a "live tail" log view (which can just poll or use server-sent-events as a lighter-weight variant).

6. Database — SQL vs NoSQL vs specialized search store

Choice: a dedicated search store (Elasticsearch/Lucene-style inverted index) for the searchable index, plus a separate document/object store for the original full documents.

Why not plain SQL with `LIKE '%word%'` : a `LIKE '%word%'` query cannot use a normal database index (a B-tree index is built for "starts with" or exact matches, not "contains this substring anywhere"). With a leading wildcard, the database has **no choice but to scan every single row and check each one** — a full table scan. Over millions or billions of documents, that is catastrophically slow, and it gets slower, not faster, as data grows.

Why an inverted index solves it: instead of scanning documents to find words, we **pre-build a map from every word to the list of documents containing it** (details in Section 7). Then a search for

"timeout" is a single fast lookup — "which documents contain timeout?" — not a scan. This is the single most important idea in this whole design.

Why keep original documents elsewhere: the inverted index is optimized for *finding* documents, not for storing every byte of them efficiently. Store the full original document (the raw log line, the full product page) in a plain document store or object storage, keyed by document id, and have the index only store enough (a snippet, key fields) to render search results and point back to the full document by id.

7. Data model (detailed)

The inverted index — the centerpiece idea. Normally we think "document -> words it contains." An inverted index flips that around: "word -> which documents contain it." That flip is *why* it's called "inverted."

Example documents:

```
doc1: "the cat sat on the mat"
doc2: "the dog sat on the log"
doc3: "the cat and the dog"
```

Inverted index (word -> posting list of doc ids):

```
"cat"  -> [doc1, doc3]
"dog"  -> [doc2, doc3]
"sat"  -> [doc1, doc2]
"mat"  -> [doc1]
"log"  -> [doc2]
"the"  -> [doc1, doc2, doc3]  (very common word, matches everything)
```

Each word's list of document ids is called a **posting list**. Looking up a word is now a single map/dictionary lookup instead of scanning every document — that's the entire speed win.

Building the index (the "analyze" step). Raw text is turned into searchable **terms** through **tokenizing** and **normalizing**:

```
"The cats SAT on mats." -> tokenize (split into words)
-> ["The", "cats", "SAT", "on", "mats"]
-> normalize: lowercase, strip punctuation, "stem" words to their
    root form (stemming: "cats" -> "cat", "mats" -> "mat")
-> ["the", "cat", "sat", "on", "mat"]
-> these normalized terms are what actually go into the inverted
    index, so a search for "CAT" also matches a document containing
    "cats".
```

Querying and ranking. A multi-word query looks up each term's posting list, then combines them (e.g. **intersect** the lists for "must contain all these words," or **union** for "any of these words"), and finally **ranks** the results by relevance rather than just returning them in arbitrary order.

Relevance ranking, explained simply (TF-IDF / BM25):

- TF ("term frequency"): a document that mentions "timeout" 5 times is probably more about timeouts than one mentioning it once.
 - IDF ("inverse document frequency"): a rare word like "checkout" is a stronger signal than a common word like "the," which is in almost every document and tells you nothing.
 - Score $\sim$ (how often the word appears in this doc) x (how rare and therefore meaningful that word is overall).
 - BM25 is the modern, refined version of this same idea, tuned to not let extremely long documents win purely by being long.
- Highest-scoring documents are returned first.

Data model summary:

Document store (source of truth, keyed by id):

doc_id	(STRING)	PARTITION KEY
fields	(JSON)	e.g. text, service, timestamp, tags

Inverted index (sharded, built FROM the documents):

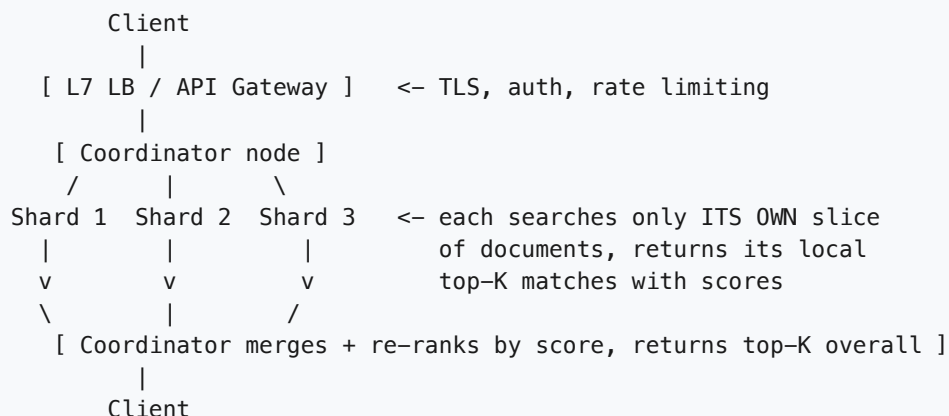
term	(STRING)	the normalized word
postings	(LIST)	[(doc_id, positions/frequency), ...]

The index is typically **sharded by document id** — each shard holds the full inverted index for *its own slice* of documents, not one shard per letter of the alphabet (more on why in Section 8).

8. Load balancer — L4 vs L7 vs API Gateway (and the scatter-gather pattern)

Choice: L7 / API Gateway at the edge, plus an internal "coordinator" pattern for the actual query fan-out — this is the special part of this design.

Because the index is split (sharded) across many nodes by document id, a single search query might need to check documents living on *every* shard. So a **coordinator node** takes the incoming query, sends it to **all** relevant shards ("scatter"), waits for each shard's local top results, and merges/re-ranks them into one final answer ("gather"). This pattern is called **scatter-gather**.



Why L7 at the edge: TLS termination, authentication, and rate-limiting the bulk-write API (so one noisy client can't overload ingestion) are classic L7 jobs. **Why the coordinator is a separate internal hop, not just the LB routing to one node:** no single shard has the whole answer, so *something* has to ask them all and combine the results — that's a job the LB itself doesn't do; it's application-level fan-out logic.

9. Caching + data flow (DB ↔ cache)

Two very different flows to describe clearly: **the index build (write) path** and the **query (read) path**.

```
BUILD PATH (ingest -> analyze -> index):
  New document arrives
    -> tokenize + normalize into terms (Section 7)
    -> append postings into an in-memory "segment"
    -> periodically (e.g. every ~1 second), flush the in-memory
        segment to disk as an immutable segment file
    -> small segment files are periodically MERGED into bigger
        ones in the background (fewer files = faster reads later)
    -> after the flush, new documents become searchable
        ("near-real-time," not instant)

QUERY PATH (scatter -> gather -> rank -> return):
  Search request -> Coordinator
    -> scatter to all shards (Section 8)
    -> each shard checks ITS OWN cache first for hot/recent
        queries before touching its on-disk segments
    -> gather each shard's top-K + scores
    -> coordinator merges, re-ranks globally, returns page of
        results
```

Caching hot queries: common searches (e.g. a popular product search term, or a log pattern many engineers search after an incident) get their *final merged results* cached at the coordinator for a short TTL. This avoids redoing the full scatter-gather for repeat queries, at the cost of results being a few seconds stale — usually a fine tradeoff.

10. Concurrency

- **Indexing while querying at the same time** is the normal state, not an edge case — this is why segments are **immutable once written**: a query never has to worry about a segment changing underneath it mid-read. New data just becomes a *new* segment; old segments are never edited in place, only eventually merged and then discarded together.
- **Segment merges** run as a background process, combining several small immutable segments into one larger one (to keep the number of files, and therefore query overhead, low). Merging reads the old segments and writes a new one; ongoing queries keep reading the old segments until the new merged one is ready, then simply switch over — no reader is ever blocked.
- **Re-indexing the same document id concurrently** (e.g. two updates racing) resolves the same way any upsert-by-id write does: last-write-wins is acceptable for most search/log use cases, since we don't need cross-document transactions.

11. Replication — needed?

Yes — every shard has replicas, for two reasons: read scale and failover.

```
Shard 1: [ Primary ] -- replicates to --> [ Replica A ] [ Replica B ]
```

- The **primary** shard accepts writes (new documents) and replicates them to its replicas.
- **Reads can be served by any replica** (or the primary), spreading query load across copies — this is the "read scale" half of the benefit.
- If the primary shard's node dies, a replica is **promoted** to primary so writes for that shard's documents can continue, and no data for that shard is lost. This is the "failover" half.
- A brand-new document is only *fully* safe once it's replicated; most systems let you choose how many replicas must confirm a write before it's considered durable (a tunable tradeoff between speed and safety).

12. Multi-geo

Region US	Region EU	Region APAC
[full index]	[full index]	[full index]
[copy, shards+]	[copy, shards+]	[copy, shards+]
[replicas]	[replicas]	[replicas]
^	^	^
users near US	users near EU	users near APAC
query locally	query locally	query locally

Replicate the whole index per region so users (or engineers searching logs) query a copy near them instead of crossing the ocean on every keystroke of a search. Writes (new documents/logs) typically land in the region closest to where they were generated and get asynchronously copied to other regions — a few seconds of lag before a document is searchable *everywhere* is normal and acceptable, matching the near-real-time expectation from Section 1.

13. Failure handling

A single shard/node goes down	-> its replica takes over serving reads and is promoted to handle writes; no full outage.
A shard is slow or briefly unreachable during scatter	-> coordinator returns PARTIAL results from the shards that DID respond, and marks the response as "partial: shard X unavailable" rather than failing the whole search.
Coordinator node itself dies	-> LB routes to another coordinator instance (they are stateless – any of them can scatter-gather).
Index segment corrupted	-> the original document store (Section 6) is the source of truth, so we can always REBUILD that segment's data by re-indexing from the source documents.
Whole region down	-> traffic reroutes to the nearest healthy region's full index copy (Section 12).

Design principle: **prefer a fast, partial, clearly-labeled answer over a slow or failed one** — a search engine returning "18 of 20 shards responded" is far better than one that hangs or errors out entirely.

14. Bottlenecks & scaling

- **Index size growing past what fits well** → add more shards; since each shard holds a slice of documents, more shards means more machines sharing the total data and query load (horizontal scaling).
- **Hot shards** (some documents/terms searched far more than others) → replicas absorb read load; very common terms can also be cached at the coordinator (Section 9).
- **Write/ingest volume** (especially for logs, per Section 2) → buffer incoming writes in a queue (e.g. Kafka) in front of the indexing nodes, so a burst of writes doesn't overwhelm indexing directly; index at a sustainable rate behind the buffer.
- **Segment merge overhead** growing with data volume → tune merge frequency/size; too aggressive merging wastes CPU/IO, too little leaves too many small segments and slows queries — a classic tunable tradeoff.
- **Retention** → for logs especially, tier old data to cheaper cold storage after N days and drop it from the fast search index, since Section 2 showed keeping everything forever in the hot tier doesn't scale.

15. Tradeoffs & what to say

"A normal database's `LIKE '%word%'` forces a full scan because there's no index structure for 'contains this substring anywhere' — so instead I build an **inverted index**: a map from every normalized word to the documents containing it, built by tokenizing and normalizing

text at write time. A query becomes fast posting-list lookups, combined and ranked by relevance (TF-IDF/BM25 — frequent-in-this-doc AND rare-overall words score highest). Because the index is too big for one machine, I shard it by document id and use a **coordinator that scatter-gathers**: fan the query to every shard, merge and re-rank the results. Each shard has replicas for read scale and failover. Writes flow through an ingest -> analyze -> index pipeline into immutable segments, which is what lets querying and indexing happen concurrently without conflicts. If a shard is unavailable, I'd rather return fast partial results, clearly labeled, than block the whole search."

Follow-ups an interviewer may probe - How would you support typo-tolerance ("fuzzy" search)? (Match terms within a small edit distance of the query term, often via an additional index structure layered on top of the exact-match inverted index.) - How do you keep ranking fast when merging results from many shards? (Each shard only needs to return its own local top-K with scores, not everything — the coordinator only merges small K-sized lists, not full result sets.) - How would you handle a query that must match an exact phrase, not just any of the words? (Store term *positions* in the postings, so "cat sat" requires "cat" and "sat" to appear at adjacent positions, not just anywhere in the same document.) - Why is this near-real-time instead of instant? (New documents sit briefly in an in-memory buffer before being flushed into a searchable segment — trading a tiny delay for much more efficient batched indexing.)

21. Metrics & Monitoring System

The problem in one line. Collect numbers like CPU usage, request latency, and error counts from thousands of servers, store them so they can be graphed over time, and fire an alert when something crosses a threshold — like a mini Datadog or Prometheus.

1. Requirements

First, a quick definition. A **metric** is just a number that changes over time — "CPU usage on server-17," "average latency of the checkout API," "how many orders failed in the last minute." A **time-series** is a stream of (timestamp, value) points for one metric — plot them on a graph with time on the x-axis and you get the classic monitoring chart.

Functional - Collect metrics from thousands of servers/services (CPU, memory, request latency, request counts, error counts, business metrics like "orders placed"). - Store every metric as a time-series so it can be queried later. - Let users **query and graph** a metric over a time range ("show me CPU on server-17 for the last 6 hours"). - Let users define **alert rules** ("if error rate > 5% for 5 minutes, page the on-call engineer") and fire alerts automatically when a rule's condition is true.

Non-functional - **Write-heavy, by a huge margin.** Thousands of machines each emit dozens of metrics every few seconds — writes vastly outnumber reads. - **High write availability** — if the monitoring system itself falls over during an outage, it fails at exactly the moment it's needed most. Never lose track of an ongoing incident. - **Fast recent-data queries** — dashboards refresh every few seconds; a query for "the last hour" must come back in well under a second. - **Alerts must not be missed and should not be needlessly duplicated** — a missed alert means nobody knows the site is down; a storm of duplicate alerts trains people to ignore them.

2. Estimation

Assume 10,000 servers, each emitting 50 metrics, once every 10 seconds.
 $\text{points/sec} = 10,000 * 50 / 10 = 50,000 \text{ points/sec}$ written
at peak (deploys, incidents cause more metrics/logging) $\approx 150,000/\text{sec}$

Each point is small: metric name + tags + timestamp + value $\approx 50\text{--}100$ bytes.
 $50,000 \text{ points/sec} * 100 \text{ bytes} = \sim 5 \text{ MB/sec} = \sim 430 \text{ GB/day}$ of RAW data

That is far too much to keep at full resolution forever, which is exactly why "downsampling" (section 6) exists: keep raw data for a few days, then compress older data into hourly/daily summaries that are 100–1000x smaller.

Read pattern: far fewer queries than writes (maybe a few thousand dashboard/API queries per second at a big company), but each query can scan many points across a time range, so reads must be efficient, not just "less frequent."

The main takeaway to say out loud: **this is a write-dominated system with a huge volume of small points**, which immediately points toward an append-optimized, time-partitioned store rather than a general-purpose relational database.

3. API style — REST vs gRPC vs GraphQL

Choice: a lightweight write/ingest protocol (often plain HTTP, sometimes a small custom binary format) for agents pushing data in; REST for the query/dashboard API.

```
INGEST (from agents, very high volume, low overhead needed):
POST /ingest
  { metric: "cpu.usage", tags: {host: "srv-17", region: "us-east"},
    timestamp: 1737100000, value: 82.3 }
(in practice, agents batch hundreds of points into one request to
cut down on network overhead)

QUERY (from dashboards/humans, much lower volume):
GET /api/v1/query?metric=cpu.usage&tags=host:srv-17&from=...&to=...
-> [ {timestamp, value}, {timestamp, value}, ... ]
```

Why not gRPC or GraphQL for ingest: ingest just needs to be as cheap as possible per point — a plain HTTP POST (or a compact binary line protocol, like Prometheus's or StatsD's) keeps agents simple and lightweight; gRPC's stronger typing isn't worth the extra complexity on thousands of tiny agents. **Why REST for queries:** a query returns one flat shape (a list of points) — there's no nested-data problem GraphQL solves, and REST's simplicity is enough.

4. Idempotency — needed?

Yes, and it's naturally easy here — this is a nice property of time-series data.

- Writing the same (metric, tags, timestamp) point twice should just **overwrite** the existing value rather than create a duplicate or add to it. Since each write is "the value at this exact timestamp is X" (not "add X to the running total"), sending it twice — say, because an agent's network request timed out and it retried — has no bad effect. This is the same idea as "set my balance to \$100" being safe to repeat.
- **Reads are naturally idempotent** — querying a time range twice just returns the same points, as long as no new data has arrived since.
- The one place duplication *can* hurt is **alerts** (see section 13) — evaluating the same rule breach twice could page someone twice. That's solved separately with alert deduplication, not with idempotent writes.

5. Network protocol

HTTP for agent-to-collector traffic; an internal message queue in between to absorb bursts.

- Agents **push** their metrics over plain HTTP (or a lightweight UDP-based protocol like StatsD, which trades perfect delivery for even lower overhead — fine for high-volume metrics where losing one point out of thousands is harmless).

- Between the collectors and the storage layer, we don't call the database directly — we put a **queue** (like Kafka) in the middle. A queue is a durable, ordered buffer: producers (collectors) drop messages in, consumers (aggregators) read them out at their own pace. This means a sudden burst of writes (say, 10,000 servers all reporting at the same second) piles up safely in the queue instead of overwhelming the database directly.
- Dashboards use normal HTTPS request/response for queries — no need for a persistent connection, since a query is just "give me these points" and done. (A live-updating dashboard can poll every few seconds, or use a lightweight subscribe mechanism — not required for the core design.)

6. Database — SQL vs NoSQL (a Time-Series Database)

Choice: a Time-Series Database (TSDB) — purpose-built stores like Prometheus's TSDB, InfluxDB, or a wide-column store like Cassandra used in a time-series pattern.

Why not a plain relational database: picture a normal SQL table with a row per data point. At 50,000+ writes per second, every write is a new row, and a B-tree index (the structure SQL uses to keep rows sorted for fast lookup) has to be updated on every single insert — that's expensive at this volume. Worse, queries here are almost always "give me all points for metric X between time A and time B" — a **range scan over time** — which a normal SQL index isn't shaped for as well as a store built around time from day one.

Why a TSDB fits: - Writes are **append-only** — you're always writing the *newest* point, never updating an old row. This is the easiest kind of write to make fast: just add to the end. - A TSDB **partitions data by time window** (e.g., one physical file/block per hour). A query for "the last 6 hours" only has to touch 6 blocks, not scan the entire dataset — this is called **time-based partitioning**, and it's the single biggest reason TSDBs are fast for this workload. - A TSDB **compresses well**: consecutive timestamps are almost evenly spaced, and consecutive values for something like CPU usage often change only slightly — specialized compression (delta-of-delta encoding for timestamps, similar tricks for values) shrinks storage dramatically compared to a generic row store.

Why not a plain key-value store: a KV store is great for single-key lookups, but "give me every point for this metric in this time range" is a *range* query, and most KV stores aren't built to make that efficient across billions of keys. A TSDB is essentially a KV store specialized for exactly this range-over-time access pattern.

7. Data model (detailed)

A metric's identity is more than just its name — it's **name + tags**. A **tag** (also called a *label*) is a **key: value** pair that lets you slice a metric — `host: srv-17`, `region: us-east`, `endpoint: /checkout`. The combination of a metric name and its exact tag values is called a **series**.

One data point:

```
metric: "http.request.latency"
tags:   { host: "srv-17", region: "us-east", endpoint: "/checkout" }
timestamp: 1737100000
value: 212.5 (milliseconds)
```

Conceptually, a wide-column layout:

series_key	(metric name + sorted tags, hashed)	PARTITION KEY
time_bucket	(e.g., which hour this block covers)	CLUSTERING KEY
timestamp	(exact second/millisecond)	CLUSTERING KEY
value	(FLOAT)	

Query "cpu.usage for srv-17, last 6 hours" ->
look up series_key, scan time_buckets covering that range -> fast,
sequential read, no full-table scan.

The cardinality problem, in one line: if you add a tag with too many unique values — like tagging every point with a random `request_id` — you create millions of unique *series* instead of a manageable few thousand, and the system slows down or runs out of memory trying to track them all. This is called **high cardinality**, and it's the #1 way people accidentally break a metrics system. The fix is a design rule: tags should have a bounded, small number of possible values (hostnames, regions, endpoint names — not unique IDs).

8. Load balancer — L4 vs L7 vs API Gateway

```

      Ingest path (push, very high volume)
Agents (10,000s)
  | HTTP, batched
  v
[ L7 LB ] -> spreads agent traffic across many collector instances
  |
[ Collectors ] (stateless, just validate + forward)
  |
[ Queue (Kafka) ] <-- absorbs bursts, durable buffer
  |
[ Aggregation / downsampling workers ]
  |
[ Time-Series Database ]

      Query path (dashboards, lower volume)
User's browser
  |
[ L7 LB / API Gateway ] <- TLS, auth, rate limiting
  |
[ Query Service ] --(cache-aside)--> [ Cache ] --> [ TSDB ]
```

Choice: L7 for both paths. On ingest, L7 lets us route by path (`/ingest` vs `/query`) to different backend pools and terminate TLS once at the edge instead of on every collector. On the query side, an API Gateway also gives us auth and rate limiting so one noisy dashboard can't overwhelm the query service. **Why not pure L4:** L4 only sees IP/port, not the HTTP path — it can't separate the ingest firehose from user-facing queries, which we want handled very differently (ingest: maximize throughput; query: minimize latency for a human waiting on a screen).

9. Caching + data flow (DB ↔ cache)

INGEST flow (write path):

Agent → Collector → Queue → Aggregation workers → TSDB
(workers do downsampling here too – see section 6/13)

QUERY flow (read path, cache-aside):

Dashboard asks for "cpu.usage, last 1 hour, host=srv-17"

→ check cache for that exact (metric, tags, time range) key

HIT → return cached points (common: many people look at the same "last hour" dashboard for the same popular service)

MISS → read from TSDB → cache with a SHORT TTL (a few seconds) → return

Why a short TTL and not a long one, unlike the URL-shortener's cache: metrics data for "the last hour" **keeps changing** — a new point arrives every few seconds — so a stale cache entry would show an outdated graph. We still cache because many users often look at the *same* popular dashboard at the *same* time (e.g., during an incident, everyone opens the same "checkout latency" graph) — that's when caching helps the most, even with a short TTL.

10. Concurrency

- **Thousands of agents writing in parallel** is the normal case, not a problem — each agent writes its own series, so there's no shared row for two agents to fight over. This is naturally "embarrassingly parallel."
- **Aggregation/downsampling workers** all reading raw points and writing rollups need to agree on how to split the work without double-processing the same range. A common approach: partition by series (e.g., `hash(series_key) % num_workers`), so each worker owns a fixed slice of series and never overlaps with another worker's slice.
- **Alert evaluation** runs continuously against fresh data. If two evaluator instances check the same rule at the same time, they could both decide "this just breached — send an alert" and double-page someone. The fix is the same **atomic claim** idea seen in other designs: only one evaluator instance "owns" a given rule at a time (via partitioning or a lock), or a dedup step downstream collapses two alerts that fire within the same short window into one notification.

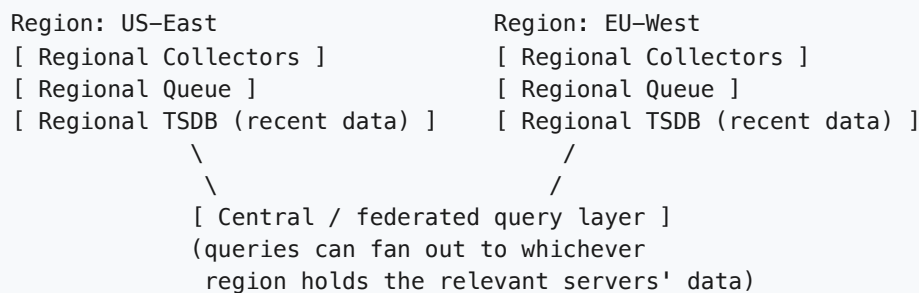
11. Replication

TSDB: replicate each time-partition to multiple nodes (e.g., 3 replicas), same idea as any distributed database — if one node dies, replicas keep serving both writes and reads for that time range.

Queue: Kafka replicates each partition across multiple brokers, so a burst of ingested points sitting in the queue isn't lost if one broker crashes before an aggregation worker reads it.

Because monitoring data is used to make real-time decisions ("is the site down right now?"), we lean toward **availability over perfect consistency** — it's fine if a graph is missing the very last few seconds of data during a node failover; it's not fine if the whole dashboard goes blank.

12. Multi-geo



Servers in a region should report to a **regional collector** first — sending every raw point across the ocean to one central store would waste bandwidth and add latency to something that's already extremely high volume. Two common approaches: **centralize** by shipping already-downsampled (smaller) data to one global store for cross-region dashboards, or **federate** by keeping raw data local per region and having the query layer fan a query out to multiple regions and merge the results, similar to a scatter-gather search. Either way, the guiding rule is the same as the estimation section: raw, high-volume data stays close to where it's produced; only compact, aggregated data travels far.

13. Failure handling

Sudden burst of writes (e.g., everyone's agents report at once, or a deploy causes a spike in error metrics)	-> the queue (Kafka) absorbs it; TSDB writers drain the queue at their own steady pace instead of being hit with the full spike directly.
An agent goes offline (server crashed, network partition)	-> that server's metrics simply stop arriving -> a GAP in that series' graph. The monitoring system itself should alert on this ("no data received from host X in 5 minutes") - a missing heartbeat is itself a useful signal.
A TSDB node dies	-> replicas (section 11) keep serving that time range; the failed node's data is rebuilt from replicas once it (or a replacement) comes back.
Alert engine evaluates the same breach twice (retry, or two evaluator instances)	-> DEDUPLICATE: if the same rule is still breaching, don't send a brand new page every evaluation cycle - track "this alert is already OPEN" and only notify again on state change (opened, or resolved).
Alert engine crashes	-> at-least-once evaluation: on restart, re-check all rules against current data rather than trusting an in-memory "already checked" list - better to risk one duplicate alert (deduped as above) than to silently miss a real incident.

Design principle, same spirit as the job scheduler: **prefer a late or slightly duplicated alert over a missed one** — the whole point of the system is to tell humans when something is wrong, so silence is the worst failure mode.

14. Bottlenecks & scaling

- **Raw ingest volume** — this is the #1 bottleneck at scale. The queue plus horizontally scaled collectors and aggregation workers (add more, each takes a slice of series) is how this scales; a single database is never asked to absorb the raw firehose directly.
- **Storage growth** — solved by **downsampling / rollups**: keep raw, full-resolution data for a short window (say, 24-48 hours, when you need to debug something *right now*), then progressively compress older data — per-minute averages after a few hours, per-hour averages after a few days, per-day averages after a few weeks. This is exactly like a photo getting blurrier the further back in time you go, and it can cut storage by 100x or more while still answering "what did last month look like on average?"
- **High-cardinality tags** (section 7) — the fix is a design/product guardrail (limit tag values, warn or reject on tags that look like unique IDs) more than a purely technical one.
- **Popular dashboards during an incident** — everyone opens the same graph at once; the short-TTL cache (section 9) absorbs this "thundering herd" of identical queries.

15. Tradeoffs & what to say

"The dominant workload is a huge volume of small, append-only writes queried by time range, so I use a Time-Series Database with time-based partitioning instead of a relational database — writes never update old rows, and range queries only touch the relevant time blocks. Agents push metrics over HTTP into a queue like Kafka, which absorbs bursts so ingest spikes don't hit the database directly; aggregation workers then downsample old data to keep storage bounded. Writes are naturally idempotent because each point sets the value at an exact timestamp rather than incrementing it. The one place duplication is dangerous is alerting, so I add explicit deduplication there — I'd rather risk one duplicate page than silently miss an incident. Regionally, raw data stays close to where it's produced, and only aggregated data travels centrally, to keep both cost and latency down."

Follow-ups an interviewer may probe - Push vs pull collection? (Push: agents send data out, simpler, works behind firewalls. Pull: the server scrapes each agent's `/metrics` endpoint on a schedule — used by Prometheus, makes it trivial to see "who stopped responding" since a failed scrape is itself a signal, at the cost of the central scraper needing to know every target.) - How do you keep an alert rule evaluation fast if it needs the last hour of data for every one of thousands of rules? (Keep a small in-memory/cache window of recent values per series for active rules, rather than re-querying the full TSDB on every evaluation tick.) - How would you support "percentile" queries (p99 latency), not just averages? (Store pre-aggregated histograms/summaries at write time — you generally cannot reconstruct an accurate p99 later from only rolled-up averages.) - What happens if the alert notification channel (e.g., paging service) itself is down? (Retry with backoff, and have a secondary channel/escalation path — an alert that can't reach anyone is as bad as one that was never fired.)

22. Ad Click Aggregator

The problem in one line. Every time someone clicks an ad, count it — accurately, in near-real-time — so advertisers get correct dashboards and correct bills, even though clicks arrive as a massive, messy, duplicate-prone firehose.

1. Requirements

First, why is this hard? Counting things sounds trivial. It gets hard because (a) the volume of clicks is huge and bursty, (b) the same click can arrive twice (a flaky network makes the browser send the same "beacon" twice), and (c) the numbers feed **billing** — an advertiser pays per click, so an inflated or duplicated count directly costs (or overcharges) real money.

Functional - Accept a firehose of **click events** — one event per ad click, from millions of users. - **Aggregate** counts: how many clicks did ad #123 get, per minute? - Show **near-real-time dashboards** to advertisers ("ad #123: 4,532 clicks in the last hour"). - Support **billing** off these same counts — advertisers are charged per click.

Non-functional - **Accuracy over raw speed.** A dashboard number that's 30 seconds stale is fine. A dashboard number that's *wrong* (double-counted, or missing clicks) is not — it's money. - **Duplicate-resistant** — a retried click beacon must not be counted twice. - **Near-real-time** — "near" means seconds-to-low-minutes of delay is acceptable; this is not a hard real-time system (unlike, say, a stock ticker). - **High throughput, bursty** — a popular ad campaign can spike traffic by 10x in seconds. - **Resistant to obvious fraud/bot clicks** (touched on, not the core focus of this design).

2. Estimation

Assume 10B ad clicks/day across the platform (a large ad network).
clicks/sec average = $10\text{B} / 86400 \approx 115,000$ clicks/sec
peak (3x average, e.g. a big shopping event) $\approx 350,000$ clicks/sec

Event size: ~200 bytes (click_id, ad_id, user_id, timestamp, metadata).
 $10\text{B/day} * 200\text{B} = 2$ TB/day of raw click events.
Keep raw events for replay/audit for 30 days $\rightarrow$ ~60 TB (cheap object storage, not a database).

Aggregate size: if there are 1M distinct ads, and we keep a count per ad per MINUTE, that's $1\text{M ads} * 1440 \text{ minutes/day} = 1.4\text{B aggregate rows/day}$.
Each row is tiny (~50 bytes: ad_id, minute, count) $\rightarrow$ ~70 GB/day.
This is FAR smaller than the raw event volume — that's the whole point of aggregating: shrink a huge stream into a small, queryable summary.

The takeaway: raw events are too voluminous to query directly for a dashboard — we must **pre-aggregate** them into small per-minute buckets, and that pre-aggregation is the heart of this design.

3. API style — REST vs gRPC vs GraphQL

Choice: a tiny HTTP "beacon" endpoint for ingest; REST for dashboard queries.

```
Ingest (called by the browser/app when an ad is clicked):  
  POST /v1/clicks { click_id, ad_id, user_id, timestamp } -> 202 Accepted  
  
Dashboard query (called by an advertiser's UI):  
  GET /v1/ads/{ad_id}/clicks?from=...&to=...&granularity=minute  
  -> [ { minute: "2026-07-17T10:00:00Z", count: 812 }, ... ]
```

Why REST/plain HTTP for ingest: the "click beacon" is fired by a browser or mobile app — it needs to be a simple, universally-supported HTTP call, nothing fancier. We reply `202 Accepted` immediately ("I've got it, I'll process it soon") rather than making the browser wait for the count to be aggregated — that would be far too slow. **Why REST for dashboard reads:** it's simple resource fetching (give me counts for this ad, this time range) — no deeply nested data (GraphQL's strength) and no streaming needed from the *client*'s point of view. **Why not gRPC for ingest:** the caller is a browser/mobile device firing a beacon — gRPC needs a compatible client and isn't a natural fit for that.

4. Idempotency — needed?

Yes — and this is the single most important idea in this whole design, because it protects billing.

Quick definition (recap): an operation is **idempotent** if doing it twice has the same effect as doing it once.

Why duplicates happen here: a mobile app on a flaky connection sends a click beacon, doesn't get a response in time, and **retries** the same beacon. Now the same real click has produced **two** HTTP requests. If we naively count both, we've double-billed the advertiser for one click. This is unacceptable — it's literally charging someone twice for the same thing.

The fix: give every click a unique `click_id`, and de-duplicate on it.

```
Client generates click_id = a UUID, once, at the moment of the click.  
If the beacon is retried, it's retried with the SAME click_id.
```

```
The aggregation step keeps track of "have I already counted this  
click_id?" (e.g., a set of recently-seen IDs, or a unique-constraint  
write). If yes -> ignore it (already counted). If no -> count it and  
remember the ID.
```

This turns "count every click event I receive" (NOT idempotent — retries inflate the count) into "count every *distinct* click_id exactly once" (idempotent — retries are harmless). We say this out loud in the interview: **at-least-once delivery of events + dedup by click_id = effectively-once counting**, which is the realistic, buildable version of "exactly once."

5. Network protocol

HTTP(S) for the ingest beacon — simple, one-shot, works from any browser/device, no persistent connection needed (a click is a single fire-and-forget event, not a stream from the client's side).

Internally, once an event is inside our system, it moves through a **message queue** (explained fully in section 6) using that queue's own protocol (Kafka's binary protocol over TCP, for example) — this is not something the outside world talks to directly.

6. Database — SQL vs NoSQL

This design actually needs **three** different storage pieces, each doing a different job. Naming them separately is the key interview move here.

- (1) The QUEUE (Kafka) — a durable, ordered, buffer for the raw click firehose. Explained in section 9's diagram. This is NOT a database in the traditional sense, but it IS where every click first lands.
- (2) OBJECT STORAGE (S3-like) — raw click events are also archived here, cheaply, for a long-window "batch recompute" if we ever suspect the real-time counts drifted from correct (see section 9, lambda/kappa).
- (3) AGGREGATE STORE — where the small, per-ad-per-minute COUNTS live, for fast dashboard reads. Choice: an OLAP / time-series / wide-column KV store (e.g., Cassandra, Druid, or a time-series DB).

Why a queue, not a database, for raw ingest: at 350K events/sec peak, writing every single raw click straight into a relational database would fall over — databases are built for queries and transactions, not for absorbing an unbounded, bursty firehose. A **message queue — explained: a durable, ordered log that many producers can write to and many consumers can read from, at their own pace** — is built exactly for "absorb a huge stream, let consumers catch up."

Why an OLAP/time-series-ish store for aggregates, not a plain relational DB: the access pattern is "give me the counts for ad #123 between time X and Y" — a **range scan** over a time-ordered key. Time-series and wide-column stores are built to make exactly this kind of query fast, and they write more cheaply at high volume than a heavily-indexed relational table would. A plain SQL table *would* technically work at smaller scale — say this tradeoff aloud — but at this volume, a store designed for time-bucketed writes is a better fit.

7. Data model (detailed)

ClickEvent (what arrives on the queue, one per click)

click_id	(STRING, UUID)	unique per click -- THE dedup key
ad_id	(STRING)	which ad was clicked
user_id	(STRING)	who clicked (for fraud checks, targeting)
timestamp	(TIMESTAMP)	when the click happened (client-reported)

Aggregate (what the stream processor produces and the store holds)

ad_id	(STRING)	PARTITION KEY -- which ad
minute	(TIMESTAMP)	the start of the 1-minute bucket
count	(INTEGER)	how many distinct clicks in that bucket

Lookup: "counts for ad_id X between time A and B" -> range scan over minute, partitioned by ad_id. This is the **ONLY** query the dashboard needs, and this table is small and fast for exactly that query.

Notice the aggregate table doesn't need `click_id` or `user_id` at all — once clicks are de-duplicated and summed into a per-minute number, the individual click's details aren't needed for the dashboard. (They're still kept in object storage for audits/fraud investigation.)

8. Load balancer — L4 vs L7 vs API Gateway

Millions of browsers/apps

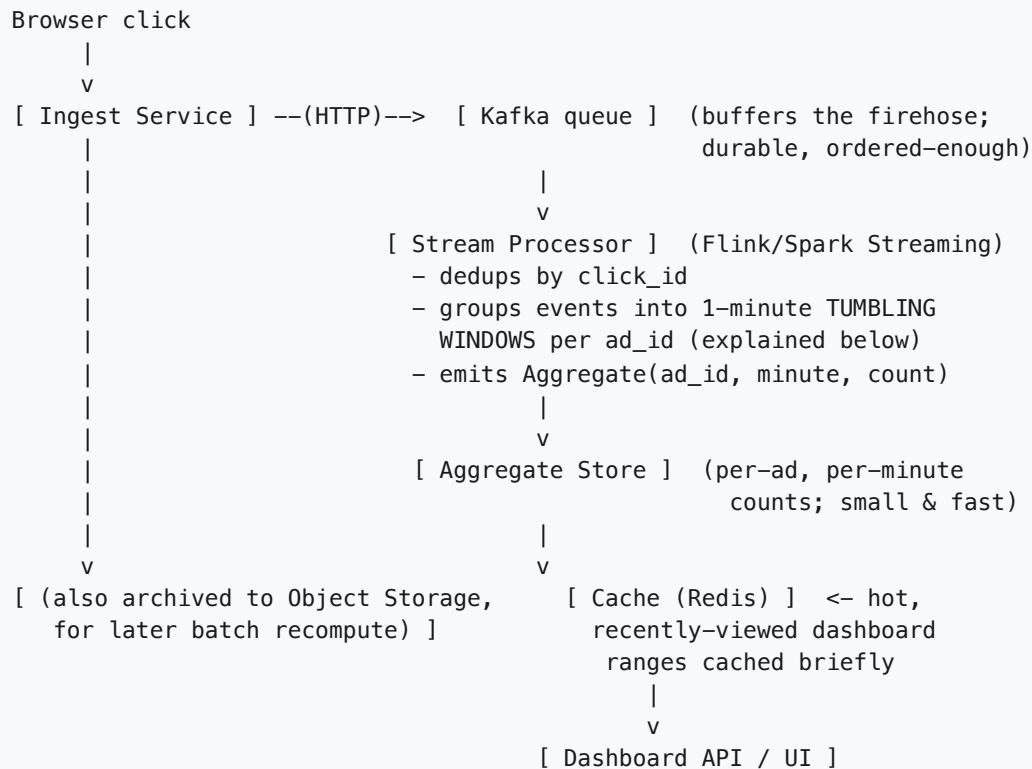
|
[L7 LB / API Gateway] <- TLS termination, basic auth/rate
| limiting on the ingest endpoint

v
[Ingest Service] (stateless, autoscaled, does almost nothing
| except validate the event and push it onto
v the queue fast)
[Kafka queue]

Choice: L7 at the edge for ingest. The ingest service is a plain stateless HTTP service, so an **L7 load balancer** gives us TLS termination and basic rate-limiting/abuse protection (a bot hammering the endpoint shouldn't be able to poison our counts or overload us). **Why not more than that:** the ingest service's *only* job is "validate the shape of this event, then hand it to the queue as fast as possible" — there's no complex routing decision to make, so a simple L7 tier in front of a thin, horizontally-scaled ingest service is all we need. The real load-spreading work is done by the **queue** (next section), not the load balancer.

9. Caching + data flow (DB ↔ cache)

This section is really "how does a click travel through the whole system," which is the centerpiece of this design.



"Stream processing," explained: instead of storing every raw event and querying it later (too slow, too much data), a stream processor reads events **as they arrive**, continuously, and keeps running totals — so "ad #123's count for the current minute" is always being updated live, without ever needing to scan the full history of clicks.

"Tumbling window," explained: a tumbling window is a fixed-size, **non-overlapping** time bucket — e.g., minute `10:00:00 – 10:00:59` is one window, `10:01:00 – 10:01:59` is the next. Every click falls into exactly one window based on its timestamp. ("Tumbling" as opposed to a *sliding* window, which would overlap — we don't need overlap here, we just want one clean number per ad per minute.)

Caching: dashboards are read far more often than the underlying minute-buckets change (an advertiser might refresh their dashboard every few seconds, but a given minute's count is finalized once and rarely re-read from the aggregate store after that). Cache recently-requested `(ad_id, time range)` results in Redis with a short TTL — this absorbs repeated dashboard refreshes without hammering the aggregate store.

10. Concurrency

The natural concurrency unit here is **per-ad-id**, and that's deliberate:

The stream processor runs many parallel WORKERS, and each worker is responsible for a SLICE of ad_ids (partitioned by ad_id, the same way Kafka itself partitions by a key). All click events for ad #123 always land on the SAME worker.

Why this matters: two workers never need to coordinate to update the same ad's count -- worker A owns ad #123 completely, worker B owns ad #456 completely. No locks, no shared mutable state, no contention. This is what lets the system scale horizontally: add more workers and more Kafka partitions, and throughput goes up roughly linearly.

Within a single worker, updating "ad #123's count for the current minute" is a simple in-memory increment (plus periodic checkpointing so a crash doesn't lose progress) — cheap and fast because it's never shared across workers.

11. Replication

Queue (Kafka):	each partition is replicated across multiple brokers (e.g., replication factor 3). If one broker dies, another already has a copy of the data -- no events are lost.
Aggregate store:	replicated the same way any database is (leader + followers, or a peer-to-peer store like Cassandra with RF=3) -- so a dashboard read survives a node failure, and writes from the stream processor aren't a single point of failure either.
Stream processor:	doesn't hold the "source of truth" itself -- its state is checkpointed (saved) periodically to durable storage, so if a worker crashes, a new one can resume from the last checkpoint instead of losing its in-progress counts.

12. Multi-geo

Region: US	Region: EU
[Ingest] -> [Kafka (US)]	[Ingest] -> [Kafka (EU)]
-> [Stream Processor (US)]	-> [Stream Processor (EU)]
-> [Regional Aggregate (US)]	-> [Regional Aggregate (EU)]
\	/
v	v
[Global Merge Job / Global Aggregate Store]	

Each region ingests and aggregates its **own** clicks locally first — this keeps latency low (a click from a user in Europe doesn't need to round-trip to the US to be counted) and can help with data-residency rules. A lightweight **global merge** step then adds up each region's per-minute counts into one global total per ad, for advertisers who want a single worldwide number. This is the same "aggregate the aggregates" idea applied one level up.

13. Failure handling

Stream processor worker crashes	-> a new worker resumes from the last CHECKPOINT and re-reads events from Kafka starting at the last committed offset. It may RE-PROCESS a few events it already handled -- but because we dedup by click_id (section 4), reprocessing is harmless. This is exactly why idempotent counting matters so much here: crashes WILL happen, and this is what keeps them from corrupting the count.
Late / out-of-order events (a click's event arrives after its minute's window "closed")	-> a click's network trip can be slow, so it might arrive after its minute's window has already been emitted. A WATERMARK -- explained: a moving marker that says "I don't expect events older than this timestamp anymore" -- tells the stream processor how long to wait before finalizing a window. Events that sneak in after the watermark passes get corrected by the batch layer below.
Traffic spike (viral ad)	-> Kafka BUFFERS the burst; the stream processor and dashboard just fall a little further behind in near-real-time, but nothing is dropped. Add more processor workers and Kafka partitions to catch up.
Streaming layer's numbers drift from being perfectly correct (rare, but possible with late data / edge cases)	-> this is where the LAMBDA ARCHITECTURE comes in, explained simply below.

Lambda architecture, explained simply: run **two** paths side by side.

FAST path (the "speed layer"): the stream processor above. Gives near-real-time counts within seconds, but might be SLIGHTLY off in rare edge cases (very late events, a brief processing hiccup).

SLOW path (the "batch layer"): periodically (e.g., once an hour or once a day), a batch job reads ALL the raw events for a time range from OBJECT STORAGE (where we archived them) and recomputes the counts from scratch, with no time pressure and no watermark guessing. This is treated as the SOURCE OF TRUTH for billing.

The dashboard shows the FAST numbers (useful, timely, "good enough"). Billing uses the SLOW, recomputed numbers once they're available (guaranteed correct). This gives users a responsive dashboard AND protects the money-critical numbers.

("Kappa architecture" is a simplification of this: instead of a *separate* batch system, you just **replay** the same stream-processing code over the archived raw events when you need a correction — one processing engine, run twice, instead of two different engines. Either is a fine answer; the key idea to say out loud is "fast approximate path + a way to recompute correctly later.")

14. Bottlenecks & scaling

- **Ingest throughput** — the ingest service is stateless and trivial (validate + push to queue), so it scales horizontally behind the L7 LB with no real ceiling.
- **Queue throughput** — Kafka scales by adding **partitions**; more partitions means more parallel consumers can read at once. Partition by `ad_id` so a single hot ad doesn't bottleneck one partition — or further hash-shard a very hot ad's events across a few partitions if one ad is truly massive.
- **Stream processor throughput** — add more worker instances; because work is partitioned by `ad_id` (section 10), this scales close to linearly.
- **Aggregate store write/read load** — writes are small (one row per ad per minute) and reads are cached (section 9), so this store rarely becomes the bottleneck; if it does, shard it the same way (by `ad_id`).
- **Hot ad campaigns** — a single viral ad can dominate traffic. Sub-partition its key (`ad_id + random_suffix`) across multiple partitions/workers, then sum the sub-partitions back together at read time — the same "split a hot key" trick used in many high-throughput systems.

15. Tradeoffs & what to say

"The core challenge isn't storing clicks — it's turning a massive, duplicate-prone firehose into small, *accurate* numbers fast enough for a dashboard, while guaranteeing the numbers that drive billing are correct. I put a message queue (Kafka) in front to absorb the bursty ingest, then a stream processor that computes per-ad, per-minute counts using tumbling windows. The single most important idea is deduplicating by a unique `click_id`, because at-least-once delivery means the same click can arrive twice — dedup turns that into effectively-once counting, which is what makes it safe for billing. I use a lambda-style split: a fast streaming path for near-real-time dashboards, and a slower batch recompute over archived raw events as the source of truth for billing, so speed and correctness don't have to fight each other. Partitioning everything by `ad_id` gives lock-free, near-linear scaling."

Follow-ups an interviewer may probe - How do you detect and filter click fraud/bots? (Rate-limit per user/IP, flag anomalous click patterns, feed a separate fraud-scoring pipeline — a whole design in itself.) - What if an advertiser needs counts per **hour** or per **day**, not just per minute? (Roll up minute-buckets into hour/day buckets, either on read or with a second aggregation pass — cheap, since the minute buckets are already small.) - How late can an event be and still count correctly? (Tune the watermark's "grace period" — longer grace period means more correctness, more delay before a window is "final.") - How do you avoid over-counting the SAME click across regions in the multi-geo design? (Each click is ingested by exactly one region — the one nearest the user — and carries one `click_id`, so the global merge step is a straightforward sum, not another dedup problem.)
